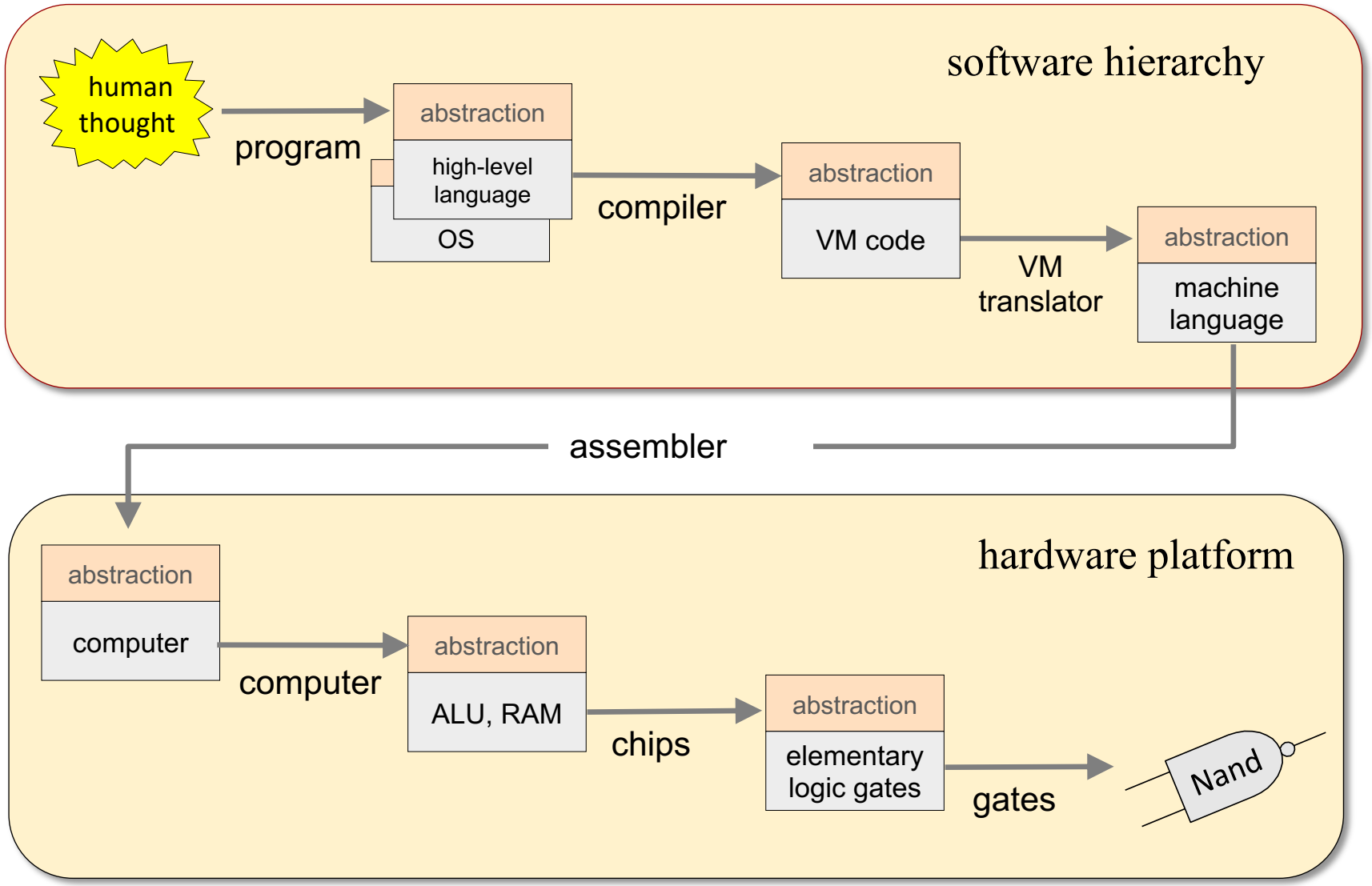
Chapter 9

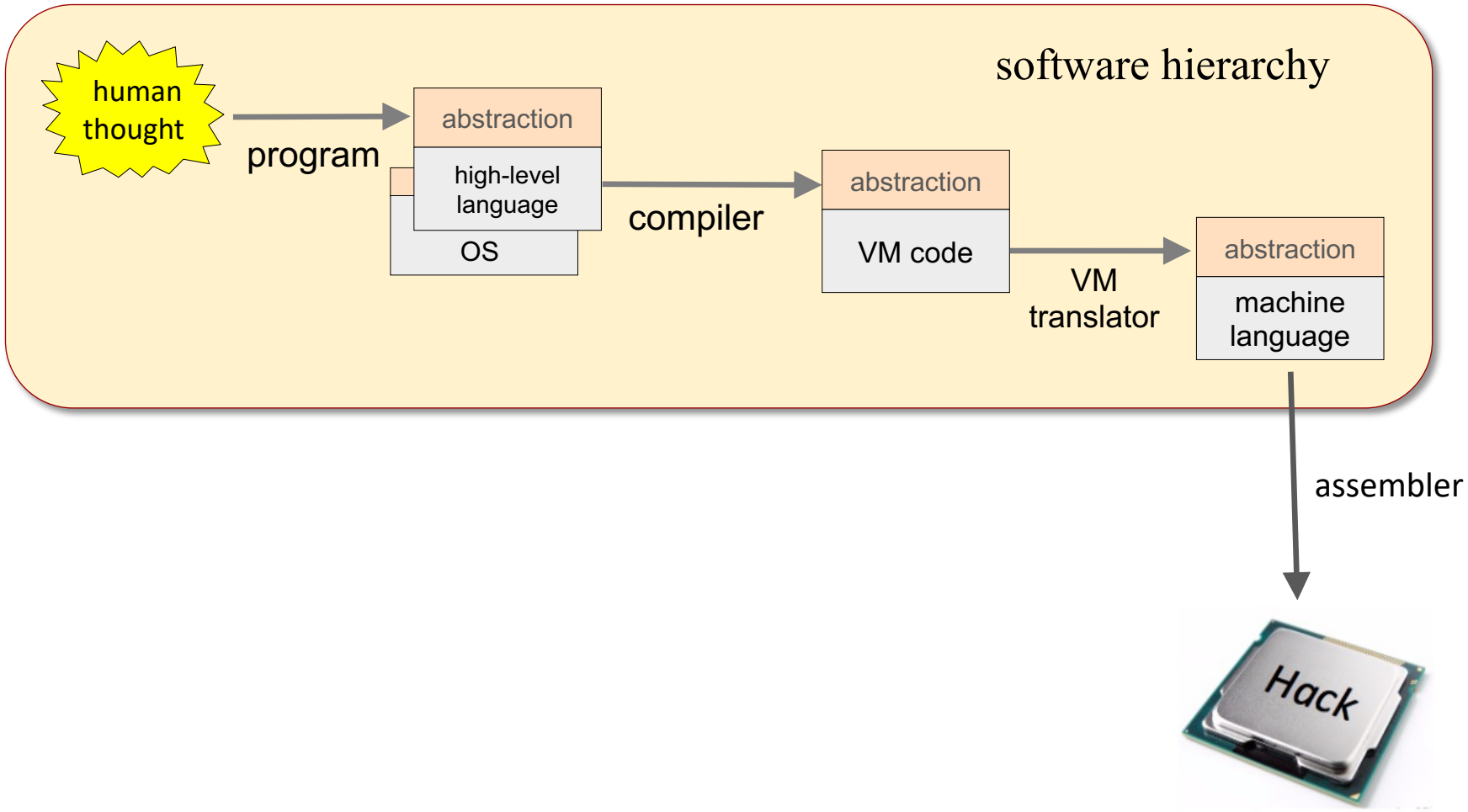
High Level Language

These slides support chapter 9 of the book
The Elements of Computing Systems
By Noam Nisan and Shimon Schocken
MIT Press

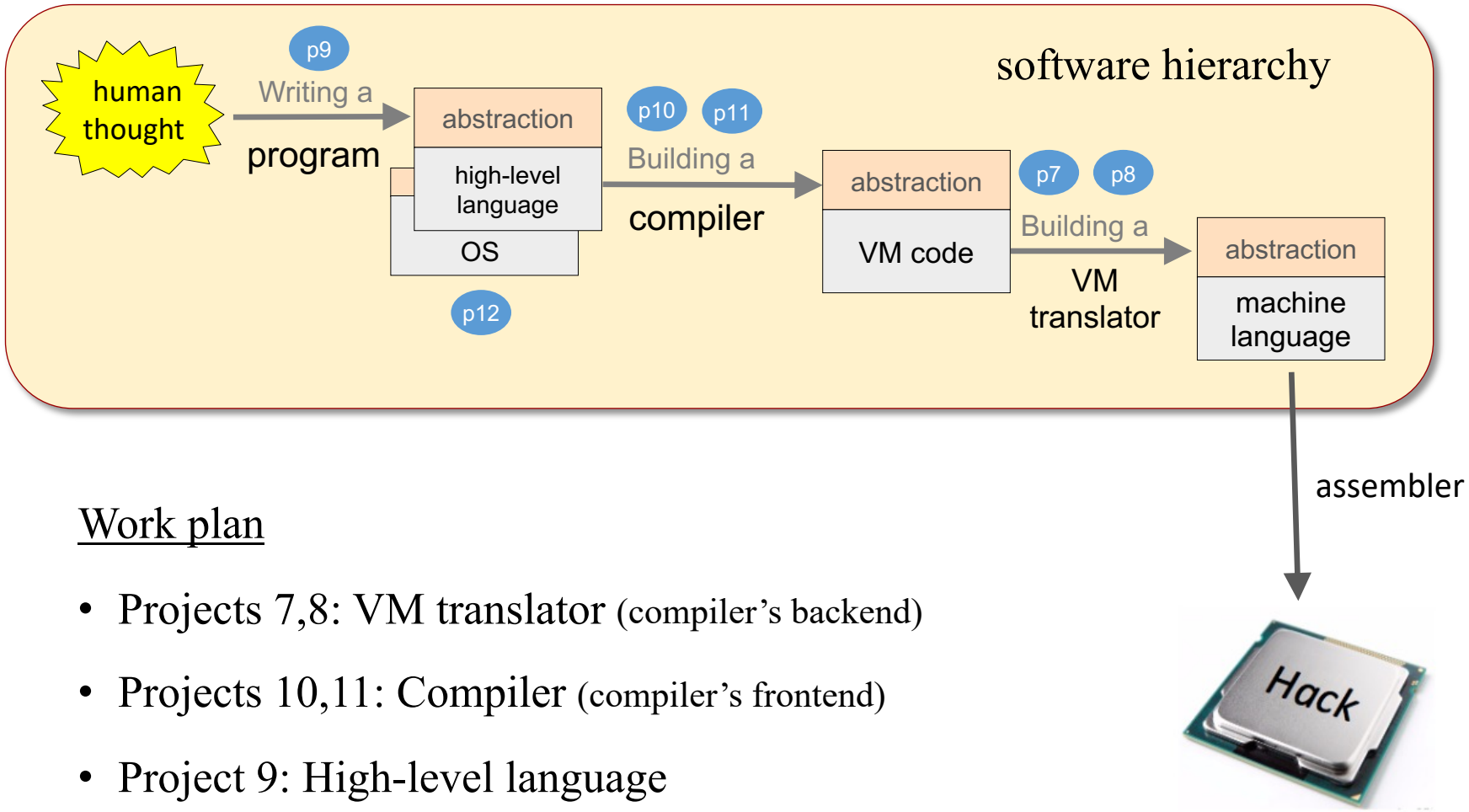
Nand to Tetris Roadmap



Nand to Tetris Roadmap: Part II



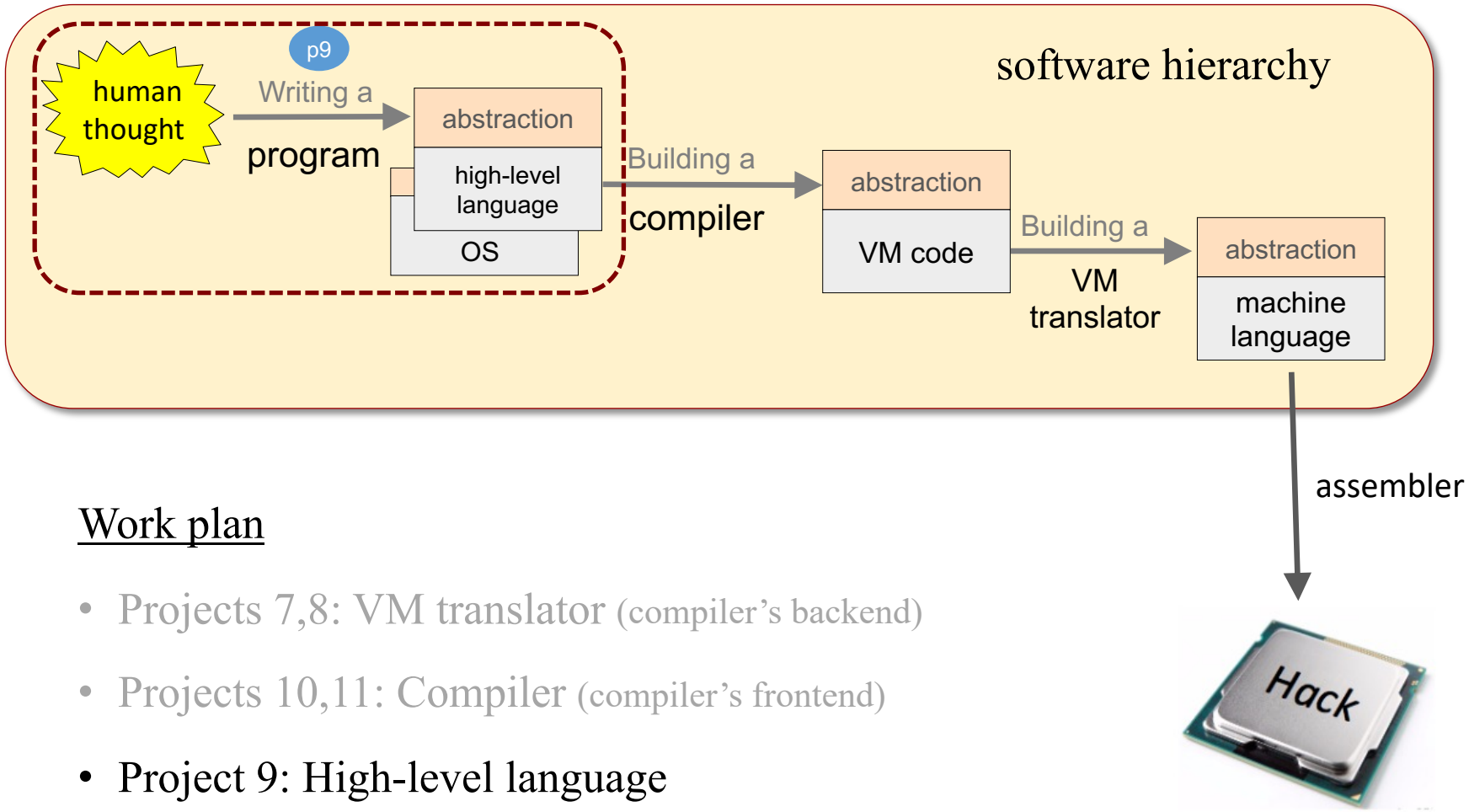
Nand to Tetris Roadmap: Part II



Work plan

- Projects 7,8: VM translator (compiler's backend)
- Projects 10,11: Compiler (compiler's frontend)
- Project 9: High-level language
- Project 12: Operating system.

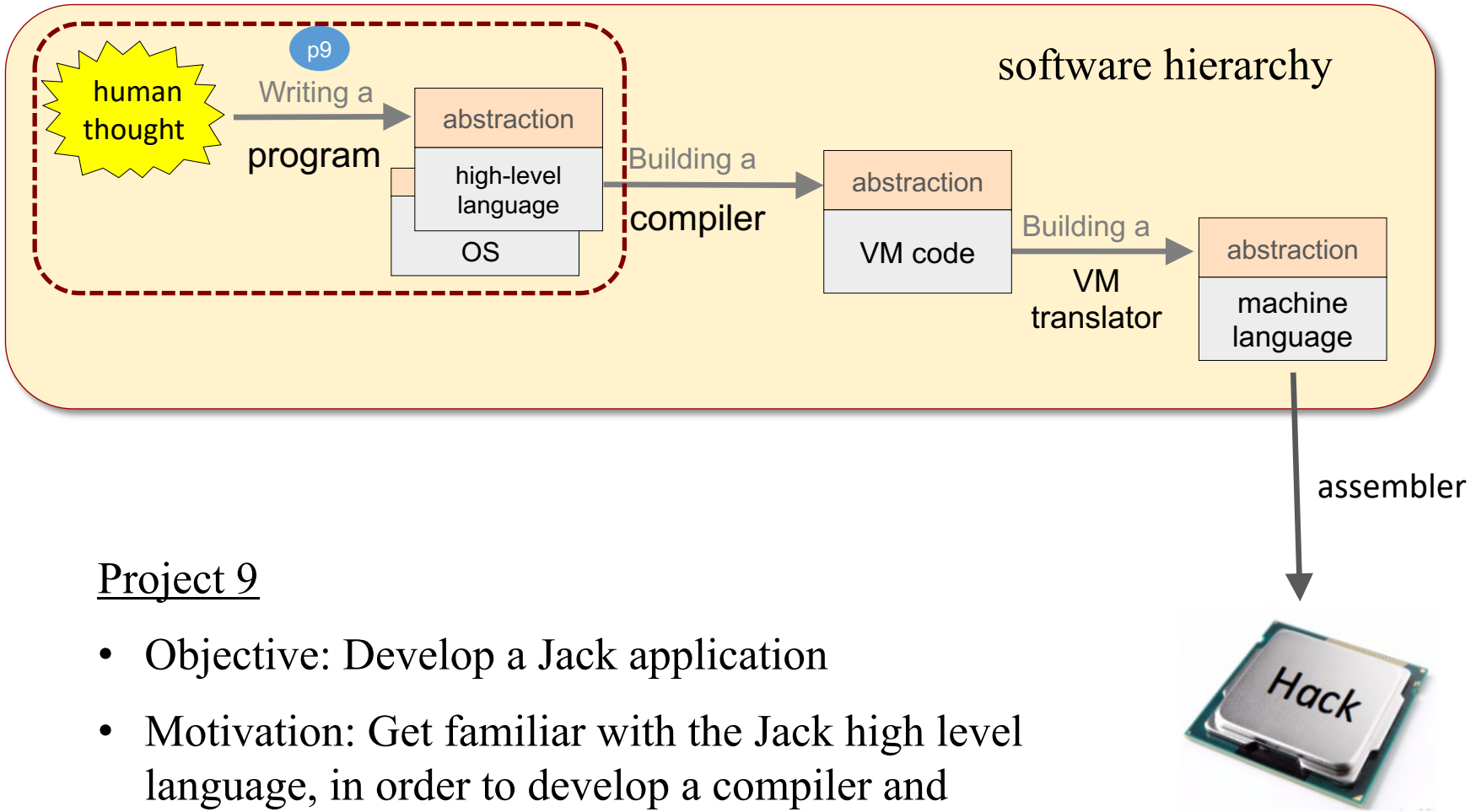
Nand to Tetris Roadmap: Part II



Work plan

- Projects 7,8: VM translator (compiler's backend)
- Projects 10,11: Compiler (compiler's frontend)
- Project 9: High-level language
- Project 12: Operating system.

Nand to Tetris Roadmap: Part II



Project 9

- Objective: Develop a Jack application
- Motivation: Get familiar with the Jack high level language, in order to develop a compiler and OS in projects 10, 11, 12.

Take home lessons

An inside view of how high-level OO languages …

- are designed
- handle primitive types and class types
- deal with strings, arrays, and lists
- create, represent, and dispose objects
- interact with the host OS

Plus, additional experience with ...

- Abstraction / implementation
- OO programming
- Application design and implementation
- Optimization.

Lecture plan

High level programming

- Program example
- Basic language constructs
- Object-based programming
- List processing

Gives a flavor of the language

Jack language specification

- The language
- The operating system

Technical reference

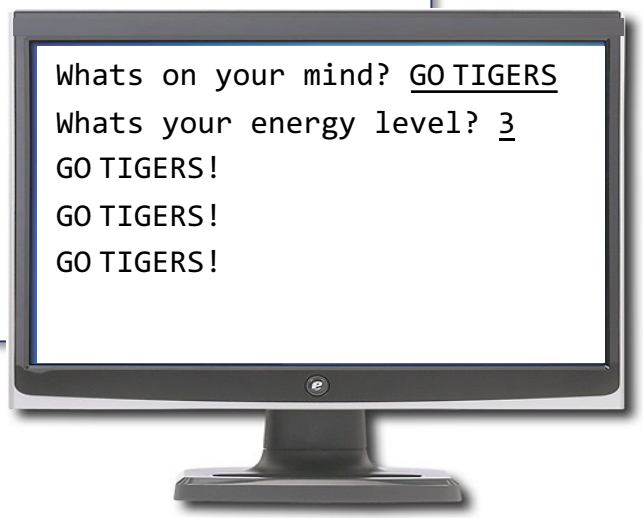
Application development

- Jack applications
- Application example
- Optimization.

Relevant to project 9

Jack program example

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```



Jack program example

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Jack:

- A simple, Java-like language
- Object-based
- Multi-purpose
- Lends itself to interactive apps
- Can be learned in an hour.

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Comments

```
// comment to end of line

/* block comment */

/** API block comment */
```

White space

(ignored)

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Program structure

- A Jack program is a collection of one or more Jack classes, one of which must be named `Main`
- The `Main` class must have at least one function, named `main`
- Program's entry point: `Main.main`

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Data types

Primitive:

- `int`
- `char`
- `boolean`

Class types:

- Built-in types, like `String`
- Programmer-defined types:
  Defined and used as needed.

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Flow of control

- `if`
- `while`
- `do`

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Input / output

- `Keyboard` (OS class):
  A library of functions for reading from the keyboard
- `Output` (OS class):
  A library of functions for writing to the screen

Lecture plan

High level programming

- ✓ Program example
- ✓ Basic language constructs
- ➡ Object-based programming
- List processing

Gives a flavor of the language

Jack language specification

- The language
- The operating system

Application development

- Jack applications
- Application example
- Graphics optimization

Basic language constructs

```
/** Performs some interaction with the user.*/
class Main {
  function void main() {
    String s;
    int energy, i;
    let s = Keyboard.readLine("Whats on your mind?");
    let energy = Keyboard.readInt("Whats your energy level?");
    let i = 0;
    let s = s.appendChar(33); // Appends the character '!'
    while (i < energy) {
       do Output.printString(s);
       do Output.println();
       let i = i + 1;
    }
    return;
  }
}
```

Simple program:

- Procedural
- One class / one function
- No objects

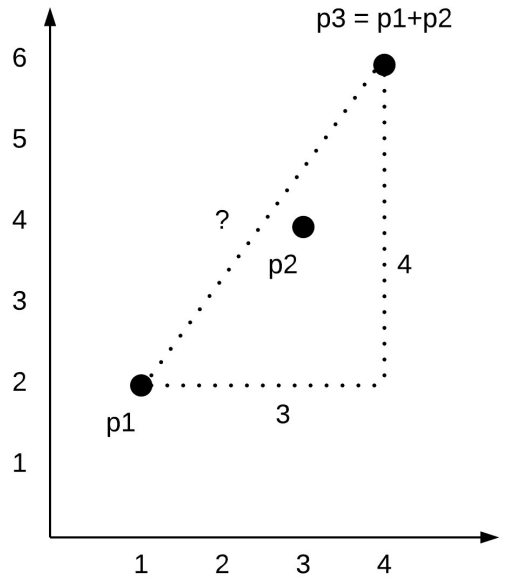
Object oriented programming

Example

We wish to represent and manipulate *points* in a two-dimensional plane.

Examples (typical program outputs):

```
(1,2) + (3,4) = (4,6)    // Vector addition
Distance betw. (1,2) and (4,6)) is 5    // Euclidean distance
...
```



Object oriented solution:

- Represent *point* as a data structure consisting of two `int` values (the point coordinates),
- Write a Jack class designed to represent and manipulate such data structures.

OO Programming

`Point` class API

```
/** Represents a point in 2D plane. */
class Point {

    /** Constructs a point from the given coordinates */
    constructor Point new(int ax, int ay)

    /** Returns the number of points constructed so far */
    function int getPointCount()

    /** Returns the sum of th
    method Poi

    /**
       a
    method int distance(Point other)

    /** Prints this point, as "(x,y)" */
    method void print() {

    // More Point methods…
}
```

Let's open the black box…

Provides services for constructing and manipulating `Point` objects

This code uses the services of the `Point` class. It can appear in any Jack class.

Client code

```
...
var Point p1, p2, p3;
let p1 = Point.new(1,2);
let p2 = Point.new(3,4);
let p3 = p1.plus(p2);
do p3.print();                              // prints (4,6)
do Output.printInt(p1.distance(p2));        // prints 5
do Output.printInt(getPointCount());        // prints 3
...
```

- Here we view the `Point` class as an *abstraction*
- The API specifies the class functionality, without describing how it delivers this functionality.
- That's all that clients need to know.

OO Programming

`Point` class

```
/** Represents a point in 2D plane. */
class Point {

   // The coordinates of this point:
   field int x, y

   // The number of point objects constructed so far:
   static int pointCount;

   /** Constructs a point and initializes it
       with the given coordinates */
   constructor Point new(int ax, int ay) {
         let x = ax;
         let y = ay;
         let pointCount = pointCount + 1;
         return this;
   }

   /** Returns the x coordinate of this point */
   method int getx() { return x; }

   /** Returns the y coordinate of this point */
   method int gety() { return y; }

   /** Returns the number of points constructed so far */
   function int getPointCount() {
       return pointCount;
   }

   // More Point methods…
```

Fields: Object properties

Statics: Class variables

Constructors:
Create and initialize objects.
A Jack class has 0 or more constructors. One of them is normally named `new`.

Methods:
Operate on the current object

Functions:
Static methods that operate on no particular object

A Jack class:

- Consists of field declarations, static variable declarations, and subroutine declarations (constructors, methods, functions)
- All fields and static variables are private
- All subroutines are public
- The only way to get access to a field or a static (from outside the class) is by calling an accessor method (like `getx`, `gety`, …)

OO Programming: Creating objects

`Point` class

```
/** Represents a point in 2D plane. */
class Point {

    // The coordinates of this point:
    field int x, y

    // The number of point objects constructed so far:
    static int pointCount;

    /** Constructs a point and initializes it
        with the given coordinates */
    constructor Point new(int ax, int ay) {
        let x = ax;
        let y = ay;
        let pointCount = pointCount + 1;
        return this;
    }

    // More Point methods…
```

Client code

```
...
var Point p1, p2;
let p1 = Point.new(1,2);
let p2 = Point.new(3,4);
...
```

A peek under the hood:

Jack programmers treat objects as abstractions

We will now preview how the object abstraction is actually implemented

This implementation will be realized when we'll build the Jack compiler and operating system.

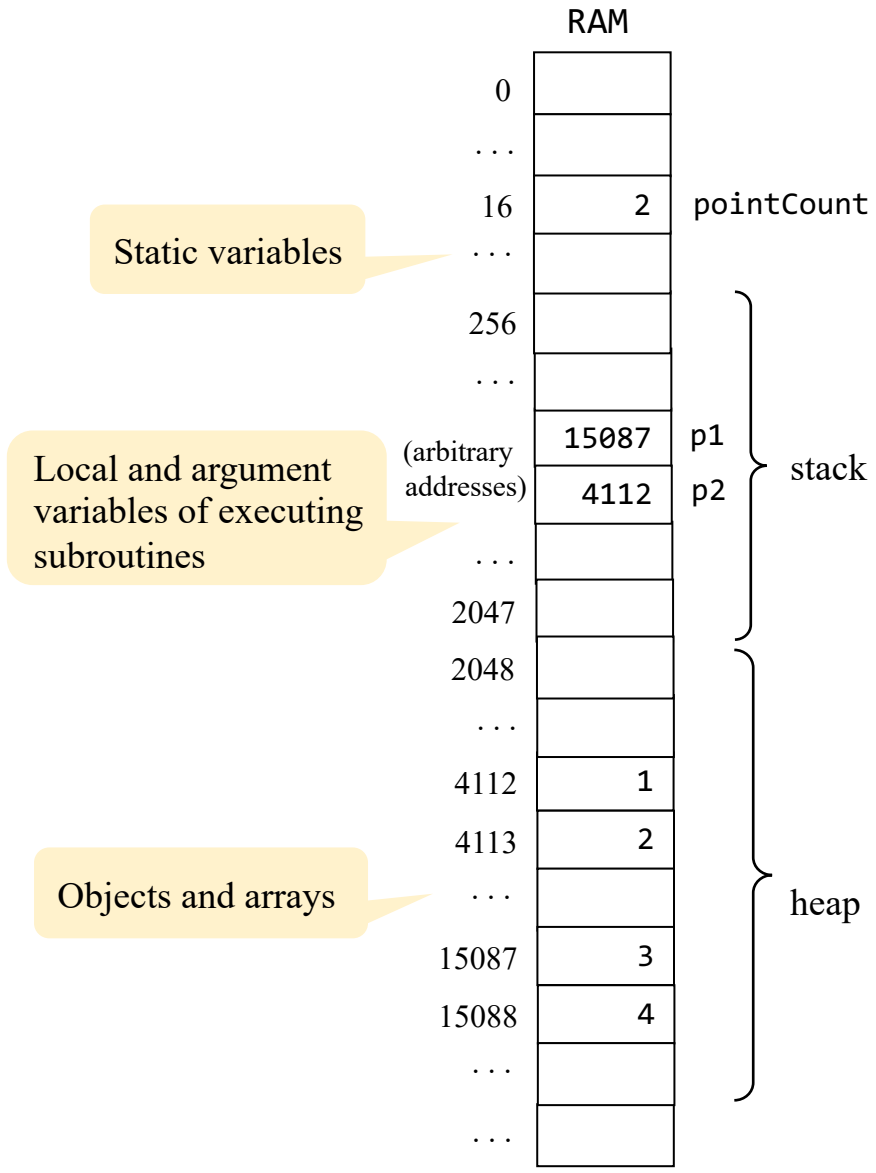
OO Programming: Creating objects

`Point` class

```
/** Represents a point in 2D plane. */
class Point {

   // The coordinates of this point:
   field int x, y

   // The number of point objects constructed so far:
   static int pointCount;

   /** Constructs a point and initializes it
       with the given coordinates */
   constructor Point new(int ax, int ay) {
         let x = ax;
         let y = ay;
         let pointCount = pointCount + 1;
         return this;
    }

    // More Point methods…
```

Client code

```
...
var Point p1, p2;
let p1 = Point.new(1,2);
let p2 = Point.new(3,4);
...
```



OO Programming: Creating objects

`Point` class

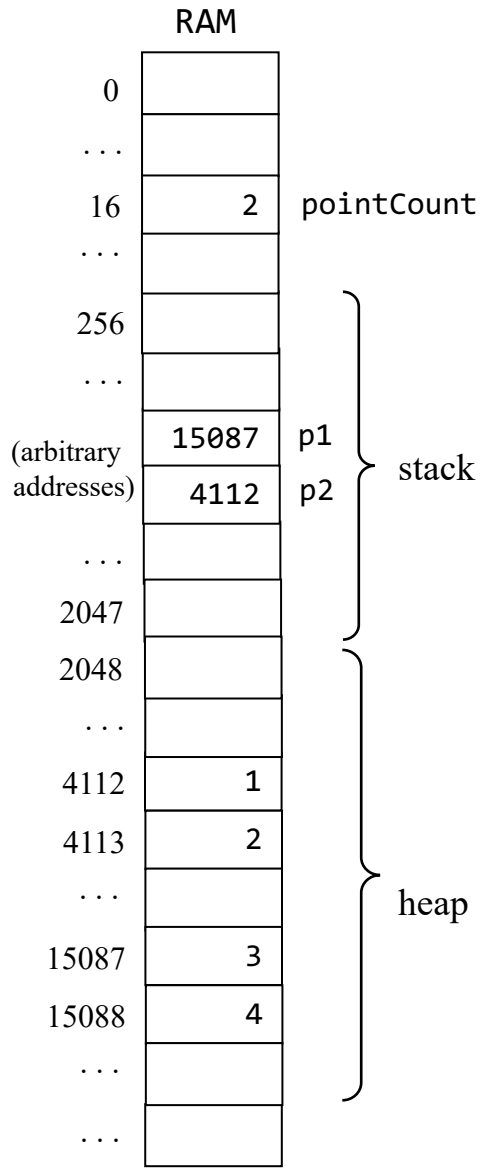
```
/** Represents a point in 2D plane. */
class Point {

   // The coordinates of this point:
   field int x, y

   // The number of point objects constructed so far:
   static int pointCount;

   /** Constructs a point and initializes it
       with the given coordinates */
   constructor Point new(int ax, int ay) {
         let x = ax;
         let y = ay;
         let pointCount = pointCount + 1;
         return this;
   }

   // More Point methods…
```

- As we will see when we'll write the Jack compiler, the *compiled* constructor's code (not shown here) calls an OS function that allocates a RAM block for representing the new object
- Next, the compiled constructor code assigns the base address of the allocated block to the object variable `this`
- And… each constructor must end with the statement `return this`

Client code

```
...
var Point p1, p2;
let p1 = Point.new(1,2);
let p2 = Point.new(3,4);
...
```

This explains how the client-side object variables (like `p1` and `p2`) end up pointing to the objects returned by the constructor.



OO Programming

`Point` class (complete class code)

```
/** Represents a point in 2D plane.
    File name: Point.jack. */
class Point {

   // The coordinates of this point:
   field int x, y

   // The number of point objects constructed so far:
   static int pointCount;

   /** Constructs a point and initializes it
       with the given coordinates */
   constructor Point new(int ax, int ay) {
         let x = ax;
         let y = ay;
         let pointCount = pointCount + 1;
         return this;
   }

   /** Returns the x coordinate of this point */
   method int getx() { return x; }

   /** Returns the y coordinate of this point */
   method int gety() { return y; }

   /** Returns the number of Points constructed so far */
   function int getPointCount() {
       return pointCount;
   }

   // Class declaration continues on top right.
```

```
   /** Returns a point which is this
       point plus the other point */
   method Point plus(Point other) {
       return Point.new(x + other.getx(),
                        y + other.gety());
   }

   /** Returns the Euclidean distance between
       this point and the other point */
   method int distance(Point other) {
      var int dx, dy;
      let dx = x - other.getx();
      let dy = y - other.gety();
      return Math.sqrt((dx*dx) + (dy*dy));
   }

   /** Prints this point, as "(x,y)" */
   method void print() {
      do Output.printString("(");
      do Output.printInt(x);
      do Output.printString(",");
      do Output.printInt(y);
      do Output.printString(")");
      return;
   }

   // More Point methods…
} // End of Point class declaration.
```

Client code

```
...
var Point p1, p2, p3;
let p1 = Point.new(1,2);
let p2 = Point.new(3,4);
let p3 = p1.plus(p2);
do p3.print();                              // prints (4,6)
do Output.printInt(p1.distance(p2));        // prints 5
do Output.printInt(getPointCount());        // prints 3
...
```

OO Programming

`Point` class – one more detail

```
/** Represents a point in 2D plane.
    File name: Point.jack. */
class Point {

   // The coordinates of this point:
   field int x, y

   // The number of point objects constructed so far:
   static int pointCount;

   /** Constructs a point and initializes it
       with the given coordinates */
   constructor Point new(int ax, int ay) {
         let x = ax;
         let y = ay;
         let pointCount = pointCount + 1;
         return this;
   }

   // All the Point subroutines we saw before, and:

   /** Disposes this point. */
   method void dispose() {
         // Calls an OS function to free the memory
         //  of this object
         do Memory.deAlloc(this);.
         return;
   }
}
```

An OS method that frees the memory space allocated to the given object

Note:

- Jack has no garbage collection
- Objects must be disposed explicitly

Best practice

- If you write a class that has a constructor, write also a dispose method
- When an object is no longer needed, dispose it.

Client code

```
...
var Point p1;
let p1 = Point.new(1,2);
...
// When the object is no longer needed:
do p1.dispose();
...
```

Lecture plan

High level programming

- ✓ Program example
- ✓ Basic language constructs
- ✓ Object-based programming
- ➡ List processing

Gives a flavor of the language

Jack language specification

- The language
- The operating system

Application development

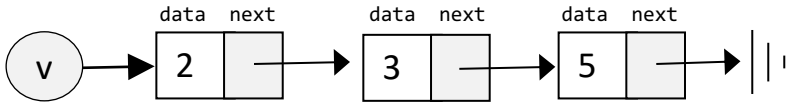
- Jack applications
- Application example
- Graphics optimization

List processing

List:

- The atom `null`, or
- An atom, followed by a list

List examples:	Abbreviated as:
`null`	`()`
`(5, null)`	`(5)`
`(3, (5, null))`	`(3, 5)`
`(2, (3, (5, null)))`	`(2, 3, 5)`



Example: the list `v = (2, (3, (5, null)))`,
commonly abbreviated as `(2, 3, 5)`

List processing

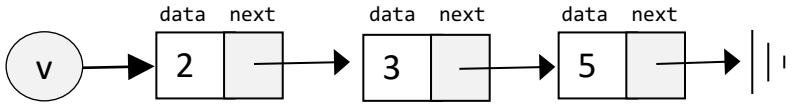
`List` class API

```
/** Represents a linked list of integers. */
class List {

    /** Creates a List */
    constructor List new(int car, List cdr)

    /** Prints this list */
    method void print() {

    /** Disposes this list */
    method void dispose() {

    // More list processing methods
}
```

Let's open the black box...

Client code

```
// Builds, prints, and disposes a list
class Main {
   function void main() {
      // Creates the list (2,3,5)
      var List v;
      let v = List.new(5,null);
      let v = List.new(3,v));
      let v = List.new(2,v));
      do v.print();    // prints (2, 3, 5)
      do v.dispose(); // disposes the list
      return;
   }
}
```



Example: the list `v` = `(2, (3, (5, null)))`,
commonly abbreviated as `(2, 3, 5)`

List processing: Creation

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   /* Creates a List. */
   constructor List new(int car, List cdr) {
      let data = car;
      let next = cdr;
      return this;
   }

   // More List methods
}
```

Client code

```
...

var List v;

let v = List.new(5,null);

let v = List.new(3, v));

let v = List.new(2, v));

...
```

List processing: Creation

`List` class

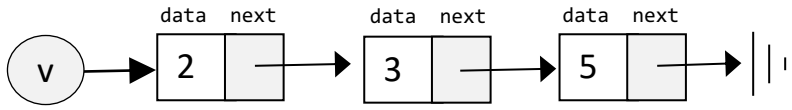
```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   /* Creates a List. */
   constructor List new(int car, List cdr) {
      let data = car;
      let next = cdr;
      return this;
   }

   // More List methods
}
```

Client code

```
...

var List v;

let v = List.new(5,null);

let v = List.new(3, v));

let v = List.new(2, v));

...
```

Same as:

```
var List v;
let v = List.new(2,List.new(3,List.new(5,null)));
```



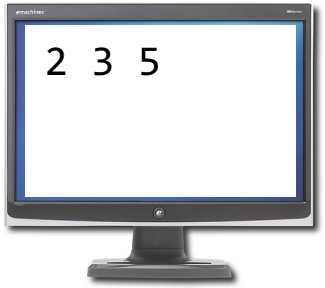
List processing: Sequential access

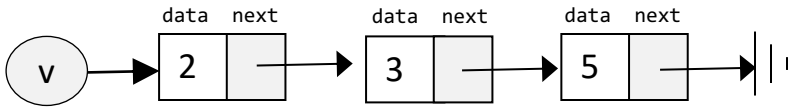
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





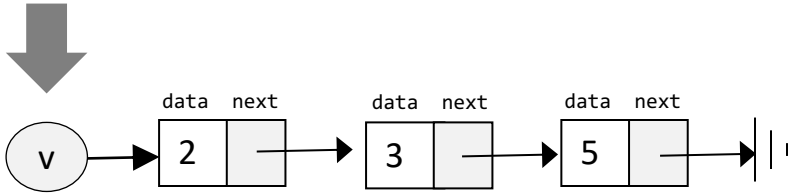
List processing: Sequential access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```



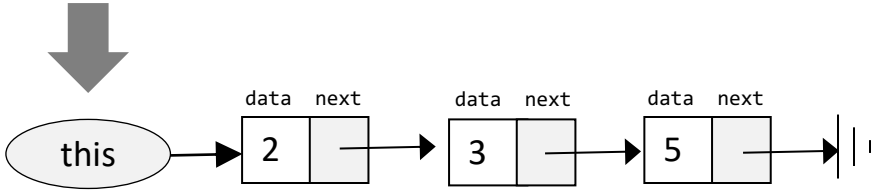
List processing: Sequential access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```



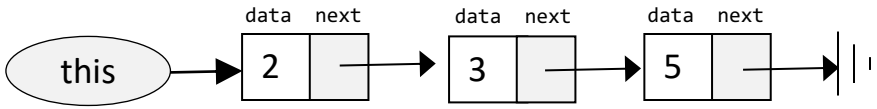
List processing: Sequential access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;     // creates a List variable and
      let current = this;   // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```



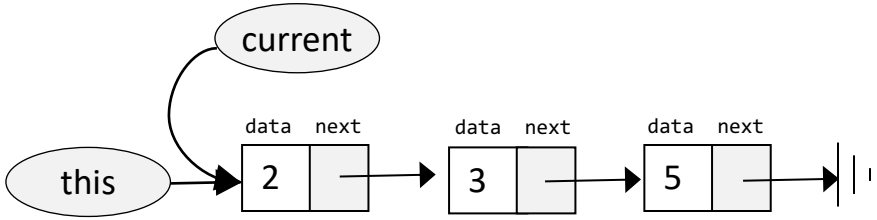
List processing: Sequential access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;     // creates a List variable and
      let current = this;   // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }
   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```



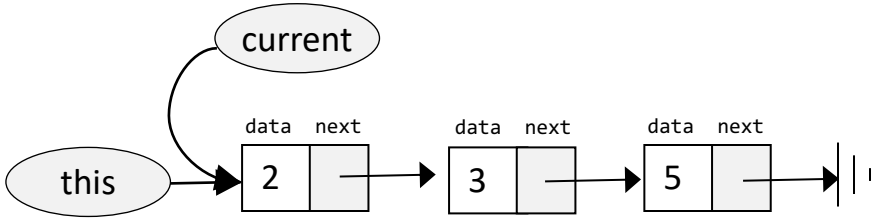
List processing: Sequential access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```



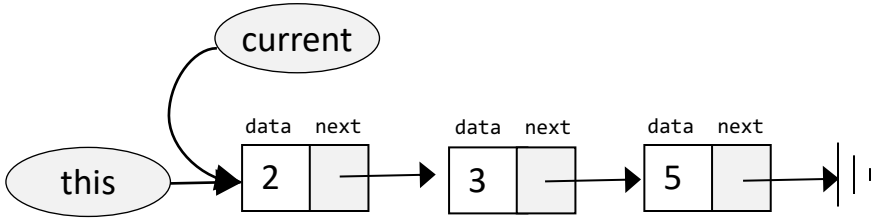
List processing: Sequential access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```



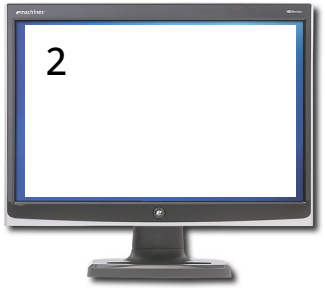
List processing: Sequential access

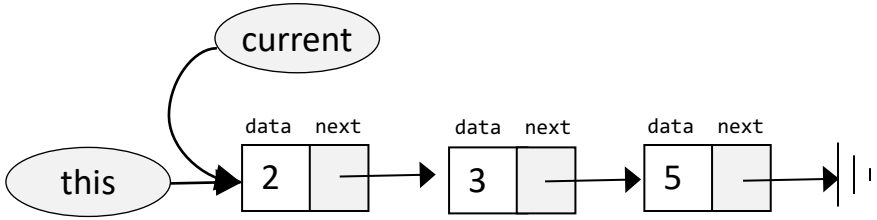
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }
   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





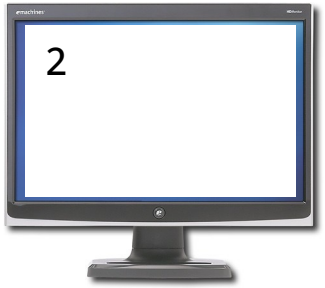
List processing: Sequential access

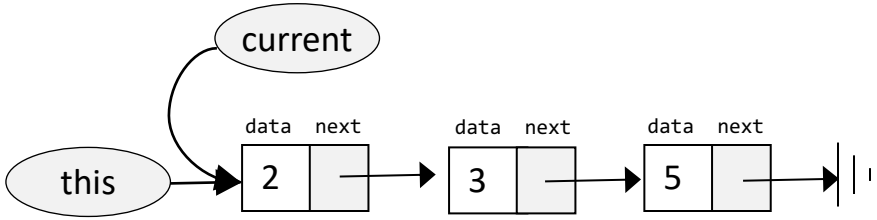
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





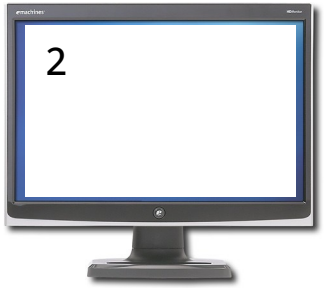
List processing: Sequential access

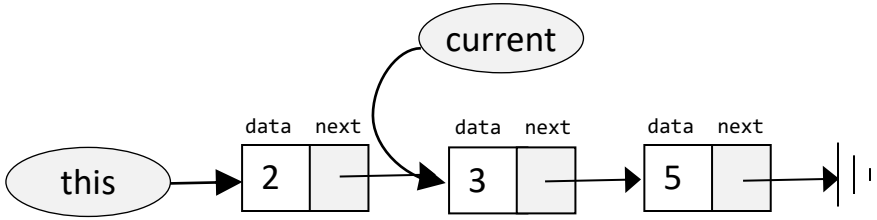
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





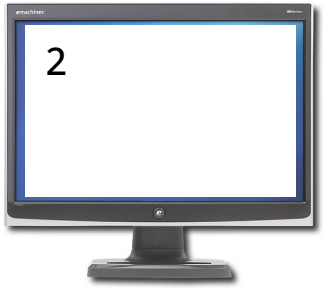
List processing: Sequential access

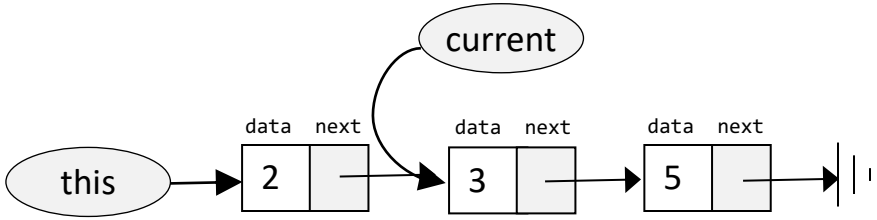
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }
   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





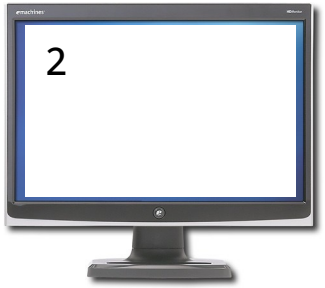
List processing: Sequential access

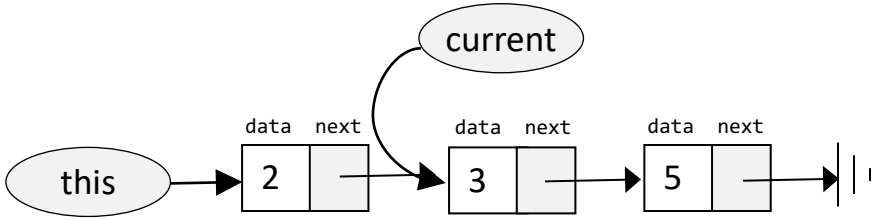
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }
   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





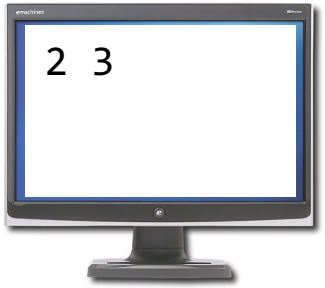
List processing: Sequential access

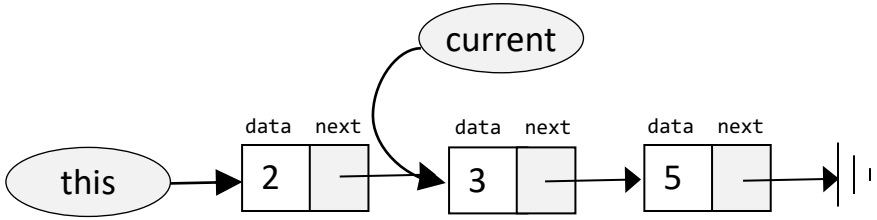
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





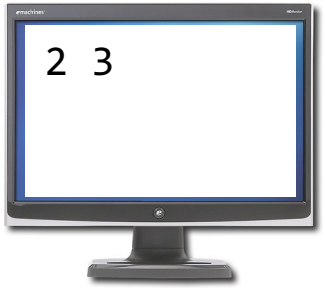
List processing: Sequential access

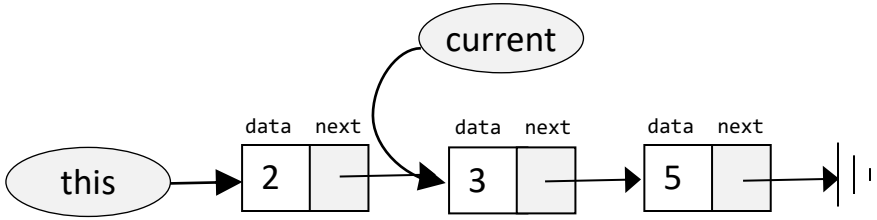
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





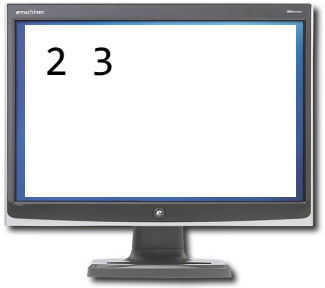
List processing: Sequential access

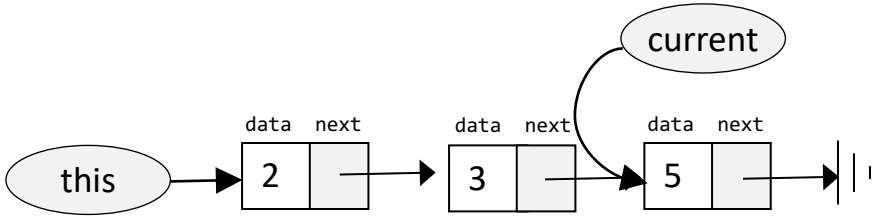
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





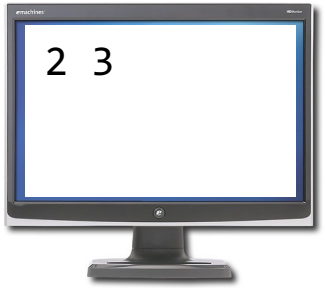
List processing: Sequential access

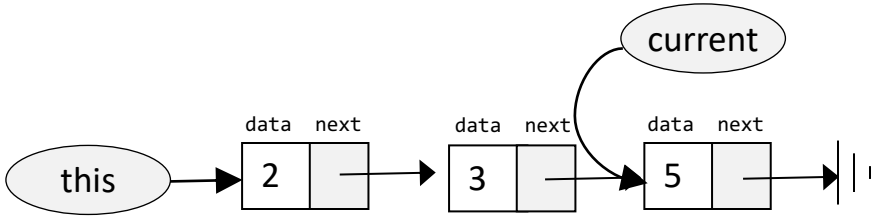
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





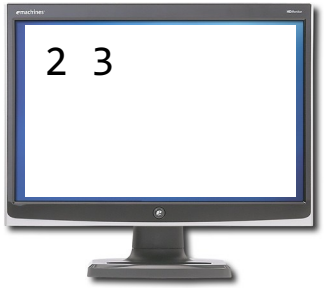
List processing: Sequential access

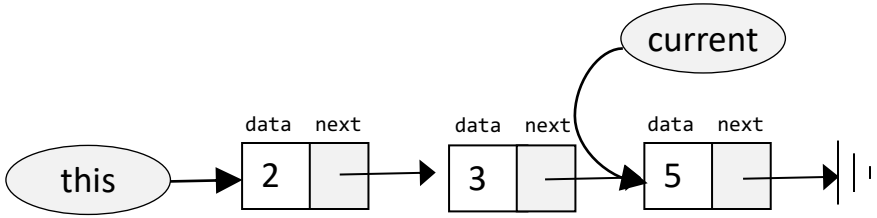
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





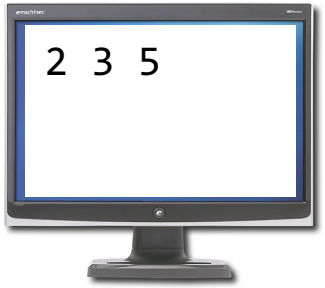
List processing: Sequential access

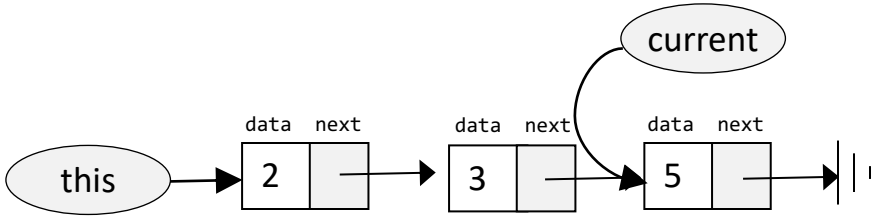
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);     // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





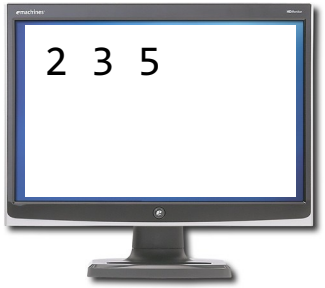
List processing: Sequential access

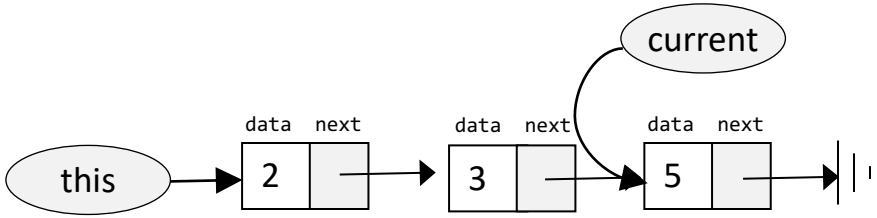
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





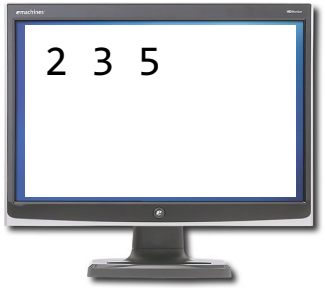
List processing: Sequential access

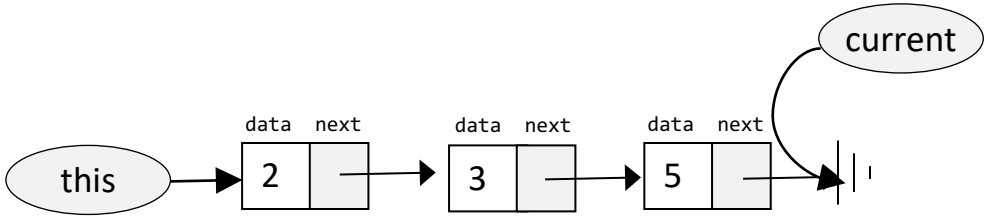
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





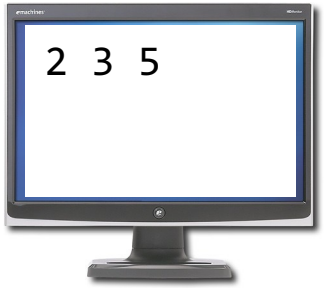
List processing: Sequential access

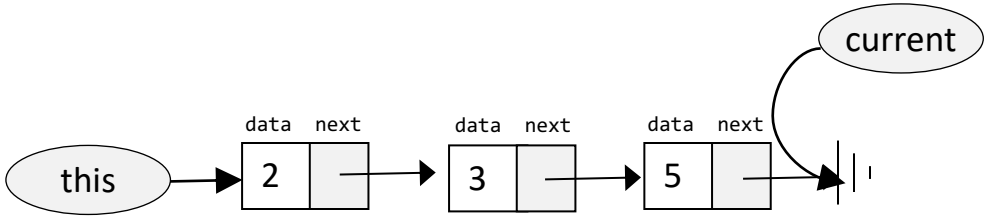
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





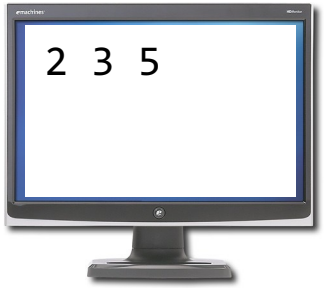
List processing: Sequential access

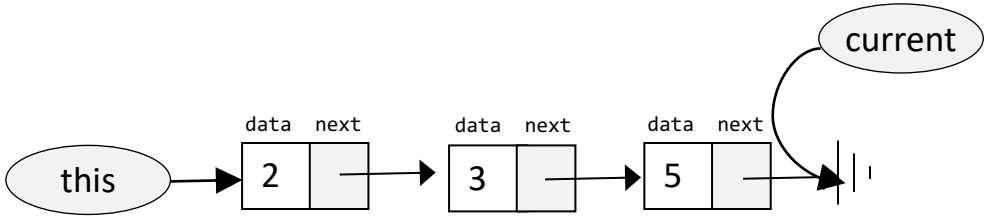
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





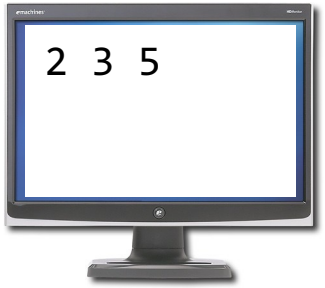
List processing: Sequential access

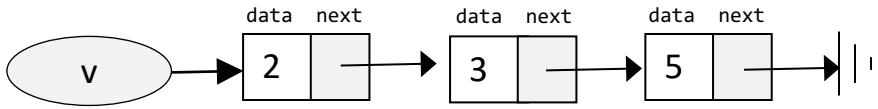
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // The constructor (code omitted)

   /** Prints this list */
   method void print() {
      var List current;    // creates a List variable and
      let current = this;  // initializes it to the first item of this list

      while (~(current = null)) {
         do Output.printInt(current.getData());
         do Output.printChar(32);    // prints a space
         do current = current.getNext();
      }
      return;
   }

   // More list methods…
```

Client code

```
var List v;
// builds the list v = (2, 3, 5)
// (code not shown)
...
do v.print();
...
```





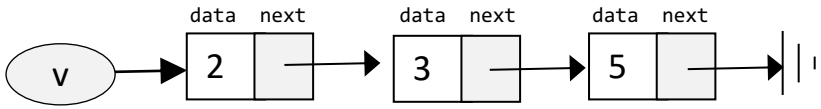
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();
...
```



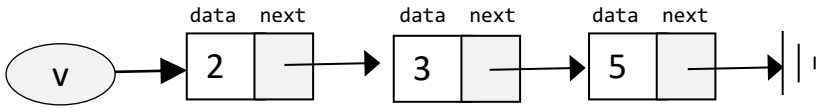
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



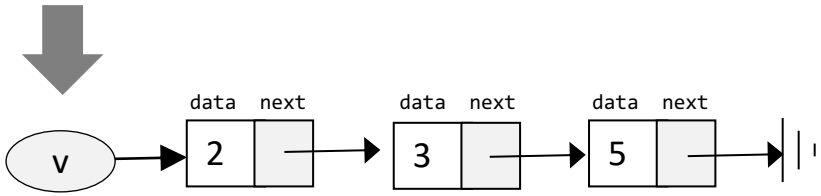
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



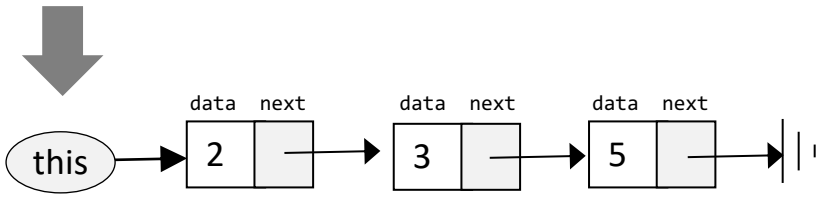
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



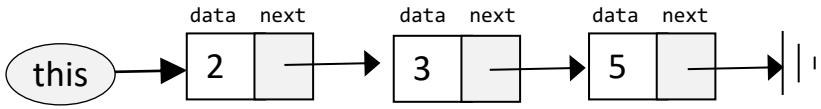
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



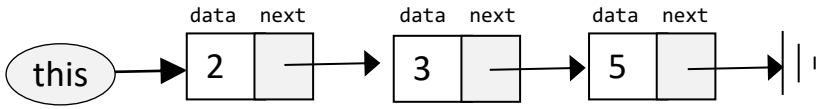
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



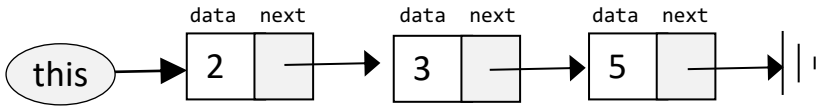
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



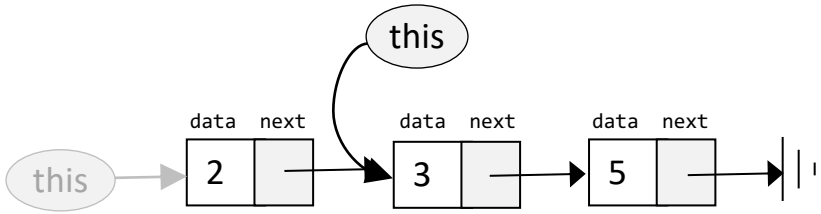
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



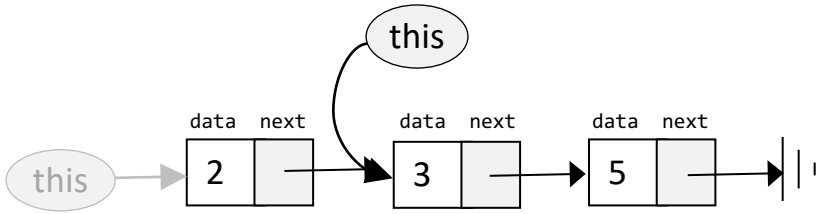
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



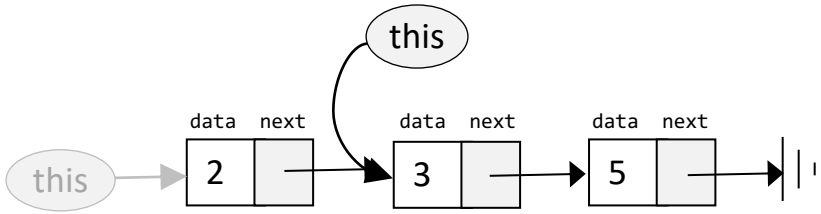
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



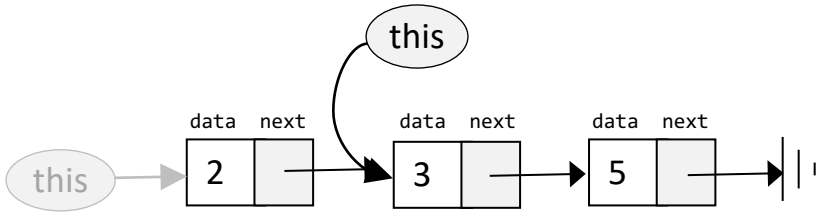
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();  return site 0
...
```



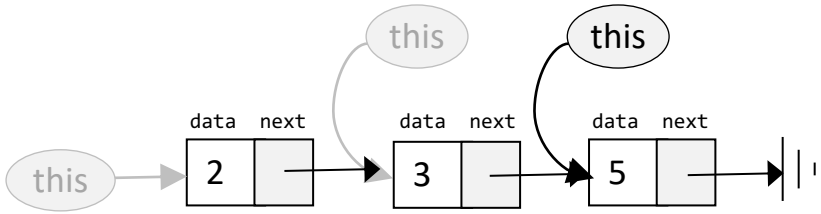
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



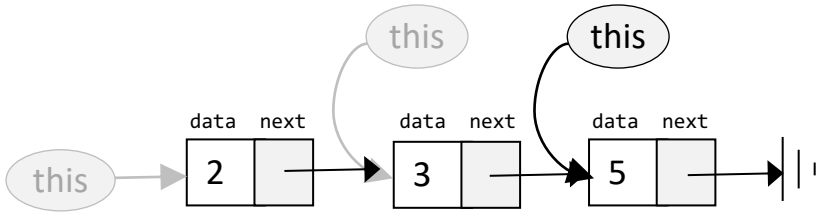
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();  return site 0
...
```



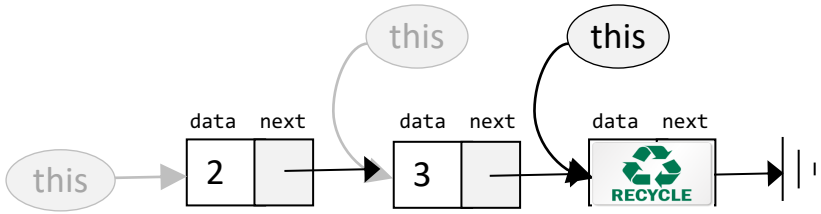
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



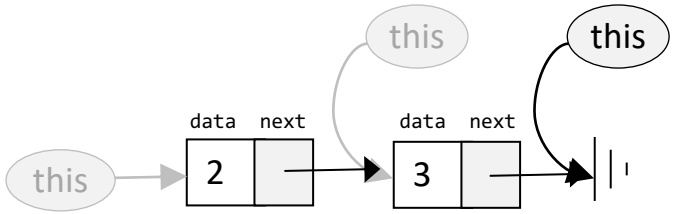
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



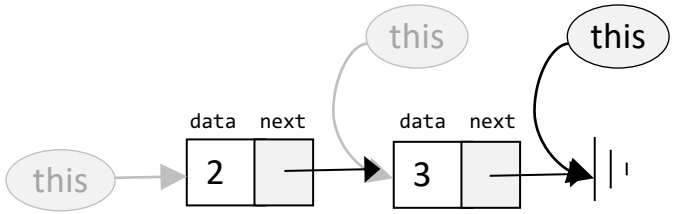
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



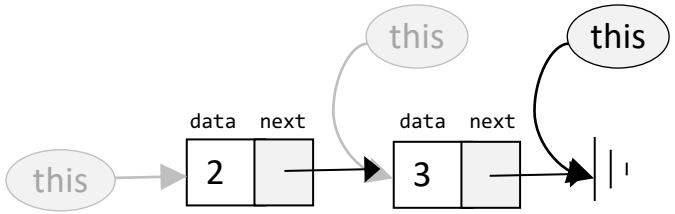
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



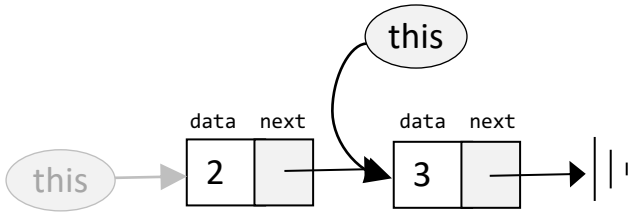
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();   return site 1
      }                       return site 2
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();   return site 0
...
```



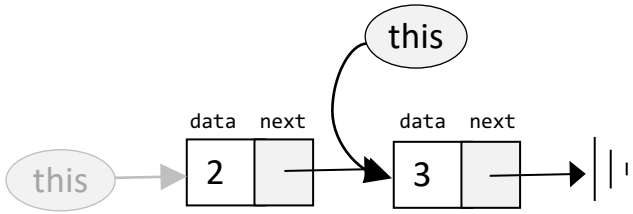
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



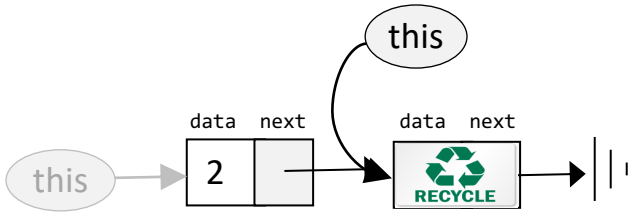
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



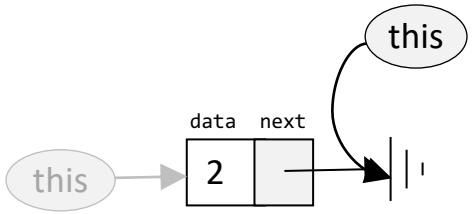
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }
}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



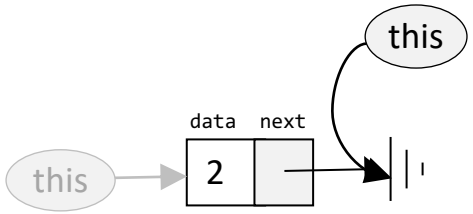
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



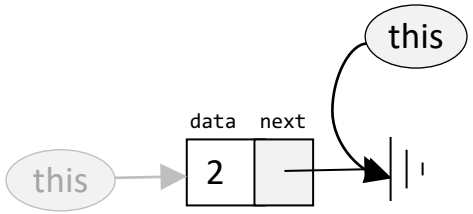
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



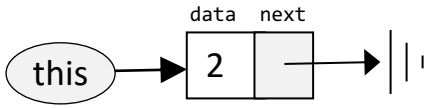
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose(); return site 1
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



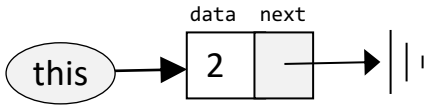
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



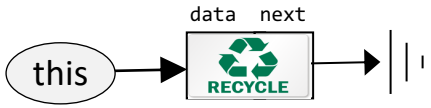
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



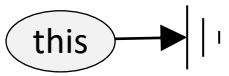
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



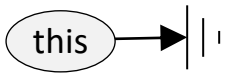
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



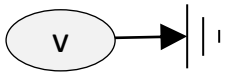
List processing: Recursive access

`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose(); return site 0
...
```



List processing: Recursive access

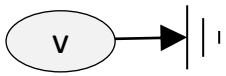
`List` class

```
/** Represents a linked list of integers. */
class List {
   field int data;      // a list consists of a data field,
   field List next;     // followed by a list.

   // Constructor and other List methods (code omitted)

   /** Disposes this list */
   // by recursively disposing its tail
   method void dispose() {
      if (~(next = null)) {
         do next.dispose();
      }
      // Uses an OS routine to recycle this list
      do Memory.deAlloc(this);
      return;
   }

}
```

Client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();
...
```

- The client code continues;
- v points to an empty list;
- The list's memory has been freed.



Lecture plan

✓ High level programming

- Program example
- Basic language constructs
- Object-based programming
- List processing

Gives a flavor of the language

Jack language specification

- The language
- The operating system

To be used as a technical reference

Application development

- Jack applications
- Application example
- Graphics optimization

Syntax

```
/** Procedural processing example */
class Main {
    /* Inputs numbers and computes their average */
    function void main() {
      var Array a;
      var int length;
      var int i, sum;
      let length = Keyboard.readInt("How many numbers? ");
      let a = Array.new(length); // constructs the array
      let i = 0;
      while (i < length) {
         let a[i] = Keyboard.readInt("Enter a number: ");
         let sum = sum + a[i];
         let i = i + 1;
      }
      ...
}
```

Syntax elements:

- White space / comments
- keywords
- Symbols
- Constants
- Identifiers

Syntax

```
/** Procedural processing example */
class Main {
   /* Inputs numbers and computes their average */
   function void main() {
     var Array a;
     var int length;
     var int i, sum;
     let length = Keyboard.readInt("How many numbers? ");
     let a = Array.new(length); // constructs the array
     let i = 0;
     while (i < length) {
        let a[i] = Keyboard.readInt("Enter a number: ");
        let sum = sum + a[i];
        let i = i + 1;
     }
     ...
}
```

Syntax elements:

- White space / comments
- keywords
- Symbols
- Constants
- Identifiers

Space characters, newline characters, and comments are ignored.
The following comment formats are supported:

```
//  Comment to end of line
/*  Comment until closing */
/** API documentation comment */
```

Syntax

```
/** Procedural processing example */
class Main {
    /* Inputs numbers and computes their average */
    function void main() {
      var Array a;
      var int length;
      var int i, sum;
      let length = Keyboard.readInt("How many numbers? ");
      let a = Array.new(length); // constructs the array
      let i = 0;
      while (i < length) {
         let a[i] = Keyboard.readInt("Enter a number: ");
         let sum = sum + a[i];
         let i = i + 1;
      }
      ...
}
```

Syntax elements:

- White space / comments
- keywords
- Symbols
- Constants
- Identifiers

`class`, `constructor`, `method`, `function`	Program components
`int`, `boolean`, `char`, `void`	Primitive types
`var`, `static`, `field`	Variable declarations
`let`, `do`, `if`, `else`, `while`, `return`	Statements
`true`, `false`, `null`	Constant values
`this`	Object reference

Syntax

```
/** Procedural processing example */
class Main {
    /* Inputs numbers and computes their average */
    function void main() {
      var Array a;
      var int length;
      var int i, sum;
      let length = Keyboard.readInt("How many numbers? ");
      let a = Array.new(length); // constructs the array
      let i = 0;
      while (i < length) {
         let a[i] = Keyboard.readInt("Enter a number: ");
         let sum = sum + a[i];
         let i = i + 1;
      }
      ...
}
```

Syntax elements:

- White space / comments
- keywords
- Symbols
- Constants
- Identifiers

() Used for grouping arithmetic expressions and for enclosing parameter-lists and argument-lists;

[] Used for array indexing;

{} Used for grouping program units and statements;

, Variable list separator;

; Statement terminator;

= Assignment and comparison operator;

. Class membership;

+ - * / & | ~ < > Operators.

Syntax

```
/** Procedural processing example */
class Main {
   /* Inputs numbers and computes their average */
   function void main() {
     var Array a;
     var int length;
     var int i, sum;
     let length = Keyboard.readInt("How many numbers? ");
     let a = Array.new(length); // constructs the array
     let i = 0;
     while (i < length) {
        let a[i] = Keyboard.readInt("Enter a number: ");
        let sum = sum + a[i];
        let i = i + 1;
     }
     ...
}
```

Syntax elements:

- White space / comments
- keywords
- Symbols
- Constants
- Identifiers

- *Integer constants* must be positive and in standard decimal notation, e.g., `1984`. Negative integers like `-13` are not constants but rather expressions consisting of a unary minus operator applied to an integer constant.
- *String constants* are enclosed within two quote (“) characters and may contain any character except newline or double-quote. (These characters are supplied by the functions `String.newLine()` and `String.doubleQuote()` from the standard library.)
- *Boolean constants* can be true or false.
- The `null` constant signifies a null reference.

Syntax

```
/** Procedural processing example */
class Main {
    /* Inputs numbers and computes their average */
    function void main() {
      var Array a;
      var int length;
      var int i, sum;
      let length = Keyboard.readInt("How many numbers? ");
      let a = Array.new(length); // constructs the array
      let i = 0;
      while (i < length) {
         let a[i] = Keyboard.readInt("Enter a number: ");
         let sum = sum + a[i];
         let i = i + 1;
      }
      ...
}
```

Syntax elements:

- White space / comments
- keywords
- Symbols
- Constants
- Identifiers

Identifiers are composed from arbitrarily long sequences of letters (A-Z, a-z), digits (0-9), and "_". The first character must be a letter or "_".

The language is case sensitive. Thus x and X are treated as different identifiers.

Variables

Variable kind	Description	Declared in	Scope
static variables	`static` *type varName*1, *varName2*, … `;` Only one copy of each static variable exists, and this copy is shared by all the object instances of the class (like *private static variables* in Java)	class declaration	The class in which they are declared.
field variables	`field` *type varName*1, *varName2*, … `;` Every object (instance of the class) has a private copy of the field variables (like *member variables* in Java)	class declaration	The class in which they are declared, except for functions, where they are undefined.
local variables	`var` *type varName*1, *varName2*, … `;` Local variables are created just before the subroutine starts running and are disposed when it returns (like *local variables* in Java)	subroutine declaration	The subroutine in which they are declared.
parameter variables	*type varName*1, *varName2*, … Used to pass arguments to the subroutine. Treated like local variables whose values are initialized "from the outside", just before the subroutine starts running.	subroutine signature	The subroutine in which they are declared.

Statements

Statement	Syntax	Description

`let`	`let` *varName* `=` *expression*`;`
	or
	`let` *varName*`[`*expression1*`]` `=` *expression2*`;`
		An assignment operation (where *varName* is either single-valued or an array). The variable kind may be *static*, *local*, *field*, or *parameter*.

`if`	`if (`*expression*`) {`
	statements1
	`}`
	`else {`
	statements2
	`}`
		Typical *if* statement with an optional *else* clause.
		The curly brackets are mandatory even if *statements* is a single statement.

`while`	`while (`*expression*`) {`
	statements
	`}`
		Typical *while* statement.
		The curly brackets are mandatory even if *statements* is a single statement.

`do`	`do` *function-or-method-call*`;`	Used to call a function or a method for its effect, ignoring the returned value.

`return`	`Return` *expression*`;`
	or
	`return;`
		Used to return a value from a subroutine.
		The second form must be used by functions and methods that return a void value. Constructors must return the expression `this`.

Expressions

A *Jack expression* is one of the following:

- A *constant*
- A *variable name* in scope. The variable may be *static*, *field*, *local*, or *parameter*
- The `this` keyword, denoting the current object (cannot be used in functions)
- An *array element* using the syntax *Arr*`[`*expression*`]`,
  where *Arr* is a variable name of type `Array` in scope
- A *subroutine call* that returns a non-void type
- An expression prefixed by one of the unary operators `-` or `~`:
  `-` *expression*: arithmetic negation
  `~` *expression*: boolean negation (bit-wise for integers)
- An expression of the form *expression op expression*
  where *op* is one of the following binary operators:
  `+ - * /` Integer arithmetic operators
  `& |` Boolean And and Boolean Or (bit-wise for integers) operators
  `< > =` Comparison operators
- `(`*expression*`)`: An expression in parenthesis

Strings

Examples:

```
...
var String s;   // Creates an object variable (pointer), initialized to null
var char c;     // Creates a primitive variable, initialized to zero
...

// Sets s to "ABC"
let s = String.new(3);
let s = s.appendChar(65);
let s = s.appendChar(66);
let s = s.appendChar(67);

// Alternatively, the Jack compiler allows:
let s = "ABC";

// Sets c to the "character", i.e. the numeric value representing 'B'
let c = s.charAt(1);
```

For more string and character operations, see the `String` class API.

Arrays

Examples:

```
...
var Array arr;
var String bla;
let helloWorld = "Hello World!"
...
let arr = Array.new(4);
let arr[0] = 12;
let arr[1] = false;
let arr[2] = Point.new(5,6);
let arr[3] = helloWorld;
...
```

Jack arrays are ...

- instances (objects) of the OS class `Array`
- not typed

A multi-dimensional array is represented as an array of arrays.

The Hack character set

key	code
(space)	32
!	33
"	34
#	35
$	36
%	37
&	38
‘	39
(	40
)	41
*	42
+	43
,	44
-	45
.	46
/	47

key	code
0	48
1	49
…	…
9	57

:	58
;	59
<	60
=	61
>	62
?	63
@	64

key	code
A	65
B	66
C	…
…	…
Z	90

[	91
/	92
]	93
^	94
_	95
`	96

key	code
a	97
b	98
c	99
…	…
z	122

{	123
|	124
}	125
~	126

key	code
newline	128
backspace	129
left arrow	130
up arrow	131
right arrow	132
down arrow	133
home	134
end	135
Page up	136
Page down	137
insert	138
delete	139
esc	140
f1	141
…	…
f12	152

Type conversions

Characters and integers can be converted into each other:

```
var char c;
// Sets c to 'A'
let c = 'A';  // Not supported by the Jack language
let c = 65;   // 'A'
// Sets c to 'A' (workaround)
var String s;
let s = "A";  let c = s.charAt(0);
```

An integer can be assigned to a reference variables, in which case it is treated as a memory address:

```
var Array arr;       // Creates a pointer variable
let arr = 5000;      // OK...
let arr[100] = 17;   // Sets memory address 5100 to 17
```

An object can be converted into an array, and vice versa:

```
var Array arr;
let arr = Array.new(2);
let arr[0] = 2; let arr[1] = 5;
var Point p;      // A Point object has two int coordinates
let p = arr;      // Sets p to the base address of the memory block representing the array [2,5]
do p.print()      // Prints "(2,5)"  (using the print method of the Point class)
```

Classes

Class declaration (stored in `Foo.jack`)

```
class Foo {
    field variable declarations
    static variable declarations
    subroutine declarations
}
```

- Jack program = collection of one or more Jack classes
- Class = basic compilation unit
- Each class `Foo` is stored in a separate `Foo.jack` file
- The class name's first character must be an uppercase letter.

Classes

Class declaration (stored in `Foo.jack`)

```
class Foo {
    field variable declarations
    static variable declarations
    subroutine declarations
}
```

- Jack program = collection of one or more Jack classes
- Class = basic compilation unit
- Each class `Foo` is stored in a separate `Foo.jack` file
- The class name's first character must be an uppercase letter.

Subroutines

Subroutine declaration

```
constructor | method | function  type  subroutineName (parameter-list) {
    local variable declarations
    statements
}
```

Jack subroutines

- Constructors: create new objects
- Methods: operate on the current object
- Functions: static methods

Subroutine types and return values

- Method and function type can be either `void`, a primitive data type, or a class name
- Each subroutine must end with the statement `return` *value*, or `return`

Subroutine calls

Subroutine call syntax: *subroutineName*(*argument-list*)

- The number and type of arguments must agree with those of the subroutine's parameters
- Each argument can be an expression of unlimited complexity.

Examples:

```
class Foo {
    ...
    method void f() {
       var Bar b;          // Declares a local variable of class type Bar
       var int i;          // Declares a local variable of primitive type int
       ...
       do g();             // Calls method g of the current class on the this object;
                           // Note: Cannot be called from within a function (static method)

       do Foo.p(3);        // Calls function p of the current class;
                           // Note: A function call must be preceded by the class name

       do Bar.h();         // Calls function h of class Bar

       let b = Bar.r();    // Calls function or constructor r of class Bar

       do b.q();           // Calls method q of class Bar on the b object.

       ...
```

End note: Some particular features of the Jack language

- The keyword `let`:
  Must be used in assignments: `let x = 0;`

- The keyword `do`:
  Must be used for calling a method or a function outside an expression: `do reduce();`

- The body of a statement must be within curly brackets, even if it contains a single statement: `if (a > 0) { return a; } else { return -a; }`

- All subroutines must end with a `return`

- No operator priority:
  - The value of this expression is unpredictable: `2 + 3 * 4`
  - To enforce operator priority, use parentheses: `2 + (3 * 4)`

- The language is weakly typed.

These language features:

- Make the life of Jack programmers a bit harder
- Make the life of Jack compiler writers much easier.

Lecture plan

✓ High level programming

- Program example
- Basic language constructs
- Object-based programming
- List processing

Gives a flavor of the language

Jack language specification

✓ The language

➡ The operating system

To be used as a technical reference

Application development

- Jack applications
- Application example
- Graphics optimization

The Jack OS

Jack code

```
// Inputs numbers and computes their average
class Main {
   function void main() {
     var Array a;
     var int length;
     var int i, sum;
     let length = Keyboard.readInt("How many numbers?");
     let a = Array.new(length); // constructs the array
     let i = 0;
     while (i < length) {
        let a[i] = Keyboard.readInt("Enter a number:");
        let sum = sum + a[i];
        let i = i + 1;
     }
     do Output.printString("The average is ");
     do Output.printInt(sum / length);
     return;
   }
}
```

Like other modern programming languages (Java, Python, C#, ...), Jack is a simple language

What makes Jack powerful is an open-ended *standard class library*

The standard class library can be seen as a language-specific OS.

The Jack OS

Jack code

```
// Inputs numbers and computes their average
class Main {
   function void main() {
     var Array a;
     var int length;
     var int i, sum;
     let length = Keyboard.readInt("How many numbers?");
     let a = Array.new(length); // constructs the array
     let i = 0;
     while (i < length) {
        let a[i] = Keyboard.readInt("Enter a number:");
        let sum = sum + a[i];
        let i = i + 1;
     }
     do Output.printString("The average is ");
     do Output.printInt(sum / length);
     return;
   }
}
```

Jack OS

- A collection of supplied Jack classes
- Similar to Java's *standard class library*.

Purpose:

- Closes gaps between high-level programs and the host hardware
- Features efficient implementations of:
  - Commonly-used functions
  - Commonly-used ADTs.

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

Two views on the OS

Abstraction (API):

How to *use* the OS — this lecture

Implementation:

How to *build* the OS — chapter 12

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
class Math {

   function int abs(int x)

   function int multiply(int x, int y)

   function int divide(int x, int y)

   function int min(int x, int y)

   function int max(int x, int y)

   function int sqrt(int x)

}
```

Usage example:

```
// Sets b to true
boolean b;
let b = (x = Math.sqrt(Math.multiply(x,x)));
...
```

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
Class String {
   constructor String new(int maxLength)
   method void   dispose()
   method int    length()
   method char   charAt(int j)
   method void   setCharAt(int j, char c)
   method String appendChar(char c)
   method void   eraseLastChar()
   method int    intValue()
   method void   setInt(int j)
   function char backSpace()
   function char doubleQuote()
   function char newLine()
}
```

Usage example:

```
// Gets the last character in the string
let c = str.charAt(str.length()-1);
...
```

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

```
Class Array {

   function Array new(int size)

   method void dispose()

}
```

Usage example:

```
// Defines an array of 10 elements
// (Jack arrays are untyped)
Array arr;
let arr = Array.new(10);
...
```

Complete OS API: book / website

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
class Output {
   function void moveCursor(int i, int j)
   function void printChar(char c)
   function void printString(String s)
   function void printInt(int i)
   function void println()
   function void backSpace()
}
```

Screen: 23 rows of 64 characters, black and white

Font: implemented by the OS `print` methods

Usage example:

```
// (Note: the ASCII code of the character a is 97)
// Prints 97, then prints the character a, twice
do Output.printInt(97);
do Output.printChar(97);
do Output.printString("a");
...
```

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
Class Screen {
   function void clearScreen()
   function void setColor(boolean b)
   function void drawPixel(int x, int y)
   function void drawLine(int x1, int y1,
                          int x2, int y2)
   function void drawRectangle(int x1, int y1,
                               int x2, int y2)
   function void drawCircle(int x, int y, int r)
}
```

Screen: 256 rows of 512 pixels,
black and white

Usage example:

```
// Draws a circle of radius 50 centered at (100,100)
// using the current color
do Screen.drawCircle(100,100,50);
...
```

The Jack OS

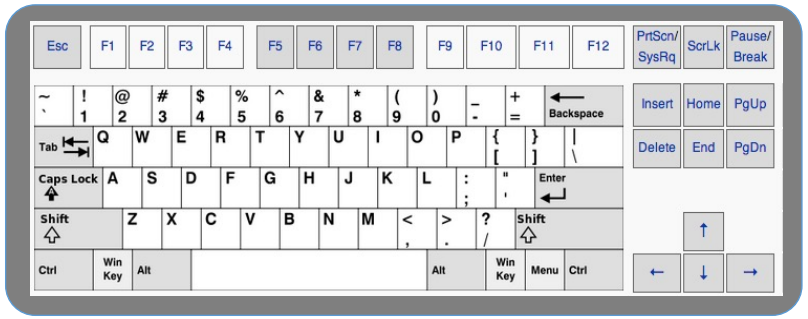
OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
Class Keyboard {
   function char keyPressed()
   function char readChar()
   function String readLine(String message)
   function int readInt(String message)
}
```

(Reads data from the standard keyboard)



Usage example:

```
...
let age =  Keyboard.readInt("enter your age: ");
...
```

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
class Memory {

   function int peek(int address)

   function void poke(int address,
                      int value)

   function Array alloc(int size)

   function void deAlloc(Array o)

}
```

Usage example:

```
// Sets RAM[257] to RAM[256]
do Memory.poke(257,Memory.peek(256));
...
```

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

```
Class Sys {

   function void halt():

   function void error(int errorCode)

   function void wait(int duration)

}
```

Usage example:

```
// Pauses for one second (1000 milliseconds)
do Sys.wait(1000);
...
```

The Jack OS

OS classes:

- `Math:` Common mathematical operations
- `String:` Common string processing
- `Array:` Used to construct and dispose arrays
- `Output:` Handles textual output
- `Screen:` Handles graphical output
- `Keyboard:` Handles input
- `Memory:` Memory management services
- `Sys:` Execution relates services

Complete OS API: book / website

Lecture plan

✓ High level programming

- Program example
- Basic language constructs
- Object-based programming
- List processing

✓ Jack language specification

- The language
- The operating system

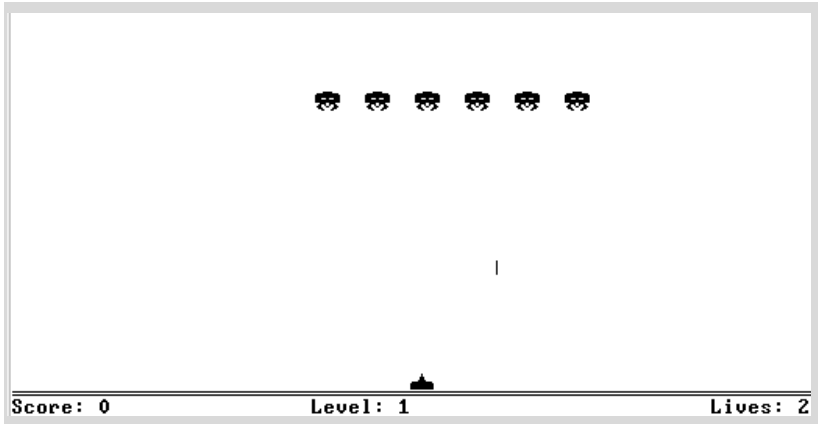
Application development

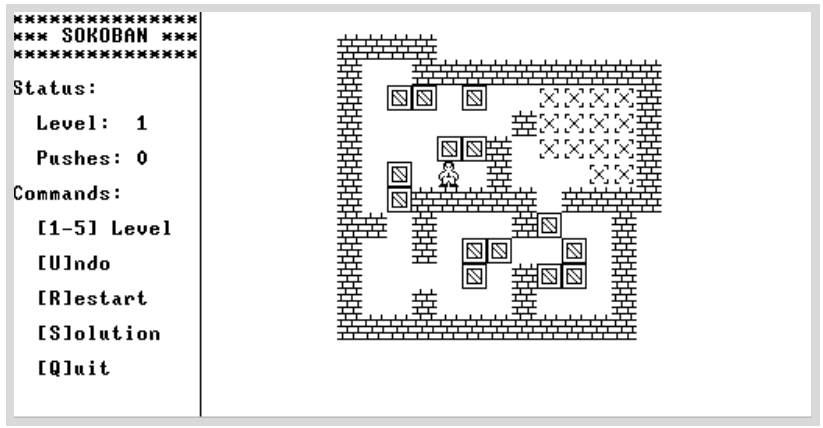
➡ Jack applications

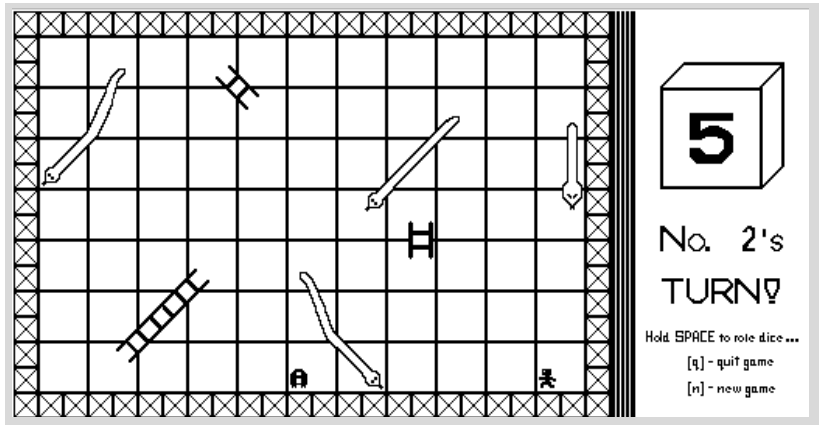
- Application example
- Graphics optimization

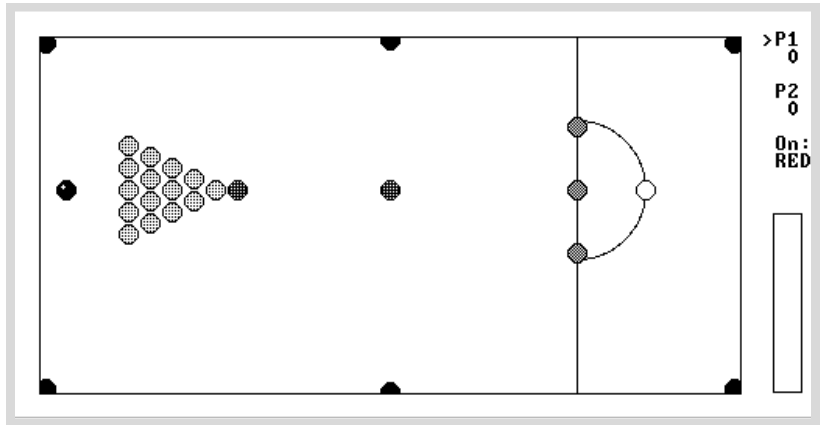
Relevant to project 9

Sample Jack applications









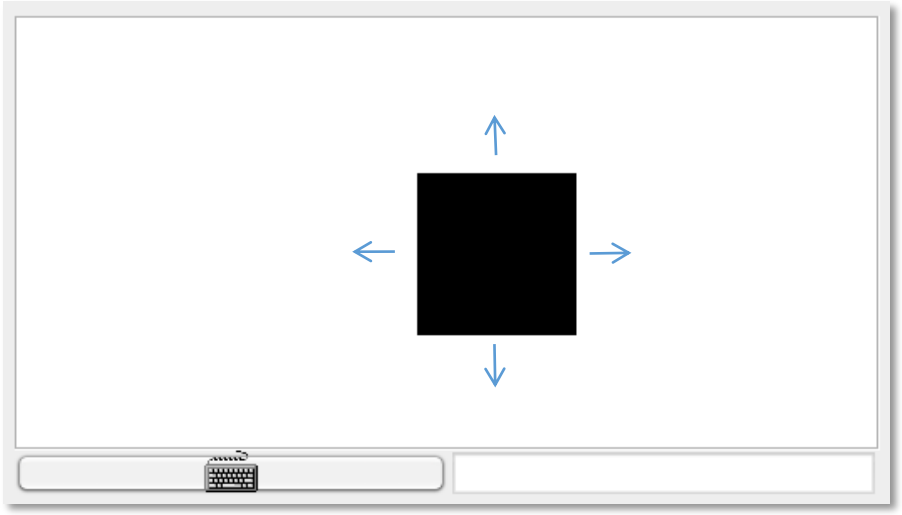
More examples: See the "cool stuff" section in `www.nand2tetris.org`

Sample Jack applications

`Square`:	A simple, interactive, multi-class OO application
`Pong`:	A complete, interactive, multi-class OO application
`Average`:	Illustrates simple array processing
`ComplexArrays`:	Illustrates various array manipulations, including two-dimensional arrays
`ConvertToBin`:	Illustrates algebraic operations, and working with `peek` and `poke`

- The source code of these programs is available in `nand2tetris/projects/11`
- These programs contain various programming tricks that may help you develop your own game in Project 9.

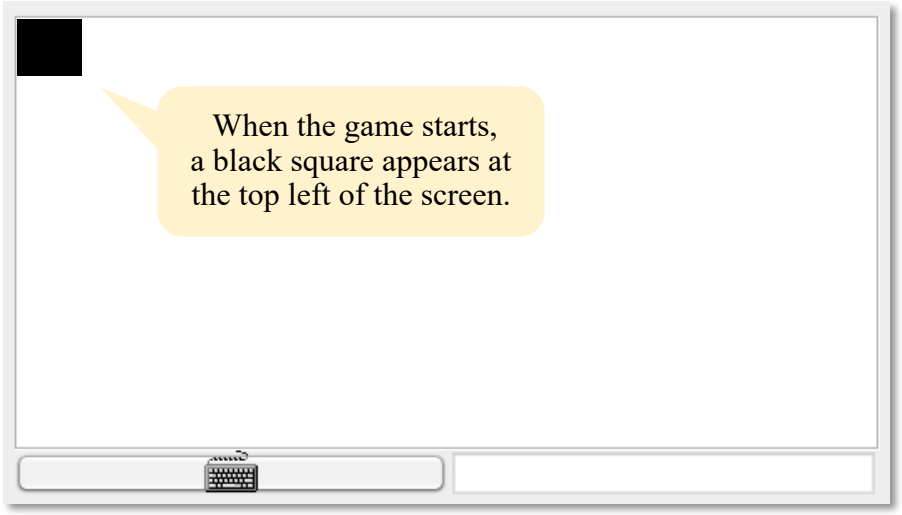
Application example: Square Dance



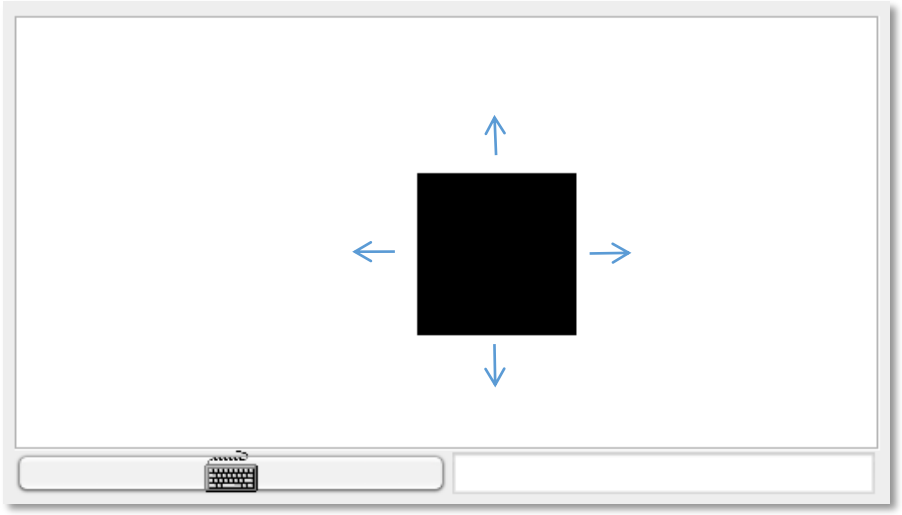
Purpose: Learning...

- OO design
- How to design / build an interactive application
- How to implement graphical objects and animation
- How to use the Jack OS

Application example: Square Dance



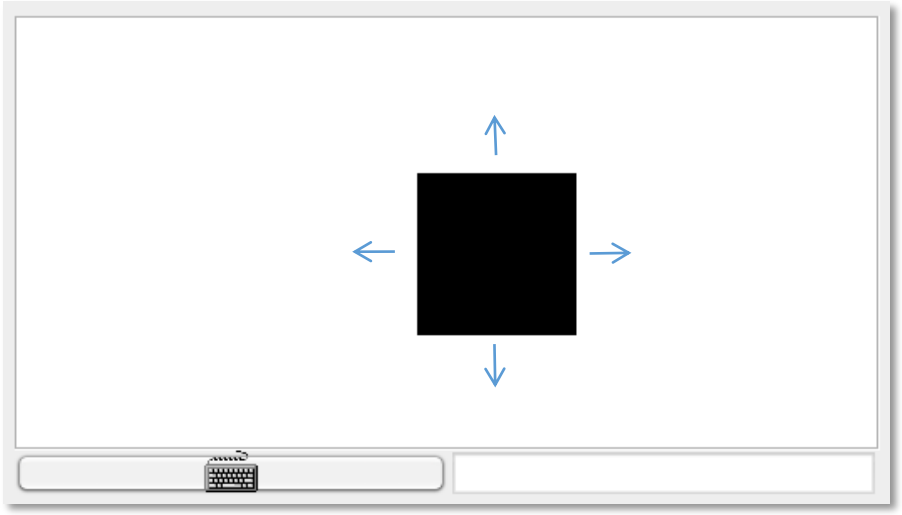
Application example: Square Dance



Usage

- up arrow: the square starts moving up, until another key is pressed
- down arrow: the square starts moving down, until another key is pressed
- left arrow: the square starts moving left, until another key is pressed
- right arrow: the square starts moving right, until another key is pressed
- x key: the square's size increases a little (2 pixels)
- z key: the square's size decreases a little (2 pixels)
- q: game over.

Application example: Square Dance



Design: 3 Jack classes:

`Square`: Represents a graphical `Square` object

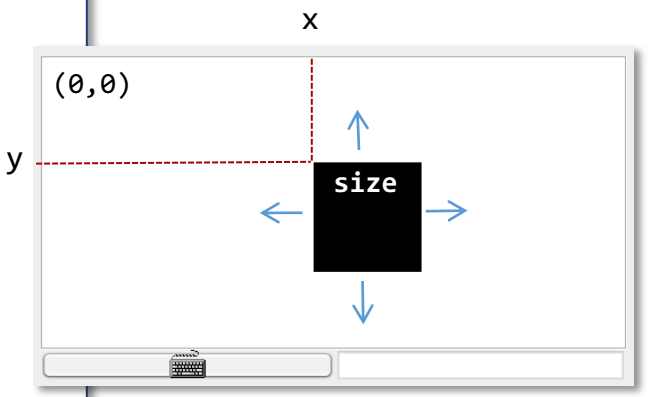
`SquareGame`: Creates a `Square`, then enters a loop that captures the user's input and *moves* / *resizes* / the square or *quits*, accordingly

`Main`: Creates a `SquareGame`, and launches the game.

Square class

Square API

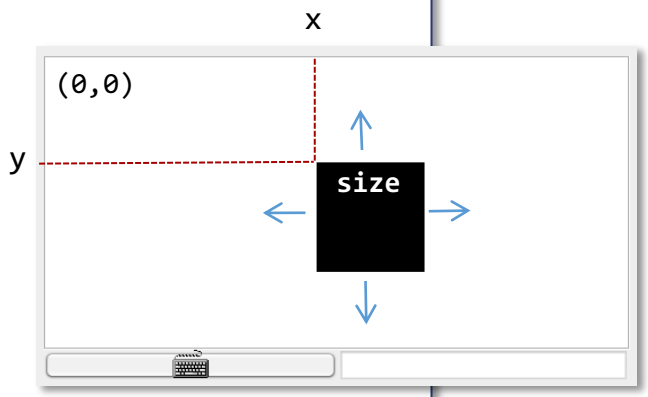
```
/** Implements a graphical square. */
class Square {

   /** Constructs a new square with a given location and size */
   constructor Square new(int Ax, int Ay, int Asize)

   /** Disposes this square */
   method void dispose()

   /** Draws this square on the screen */
   method void draw()

   /** Erases this square from the screen */
   method void erase()

   /** Increments this square's size by 2 pixels */
   method void incSize()

   /** Decrements this square's size by 2 pixels */
   method void decSize()

   /** Moves this square up by 2 pixels */
   method void moveUp()

   /** Moves this square down by 2 pixels */
   method void moveDown()

   /** Moves this square left by 2 pixels */
   method void moveLeft()

   /** Moves this square right by 2 pixels */
   method void moveRight()
}
```



Square class

Square.jack

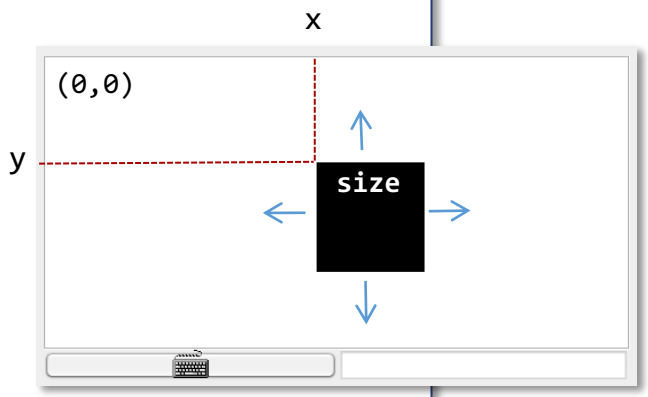
```
/** Implements a graphical square. */
class Square {

   field int x, y; // screen location of the square's top-left corner
   field int size; // length of this square, in pixels

   ...

   /** Draws the square on the screen. */
   method void draw() {
      do Screen.setColor(true);
      do Screen.drawRectangle(x, y, x + size, y + size);
      return;
   }

   /** Erases the square from the screen. */
   method void erase() {
      do Screen.setColor(false);
      do Screen.drawRectangle(x, y, x + size, y + size);
      return;
   }

   ...
```



Square class

Square.jack

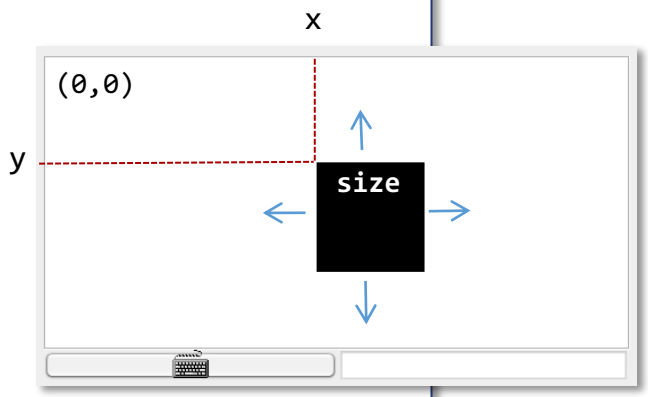
```
/** Implements a graphical square. */
class Square {

   field int x, y; // screen location of the square's top-left corner
   field int size; // length of this square, in pixels

   ...

   /** Increments the square size by 2 pixels. */
   method void incSize() {
      if (((y + size) < 254) & ((x + size) < 510)) {
         do erase();
         let size = size + 2;
         do draw();
      }
      return;
   }

   /** Decrements the square size by 2 pixels. */
   method void decSize() {
      if (size > 2) {
         do erase();
         let size = size - 2;
         do draw();
      }
      return;
   }

   ...
```



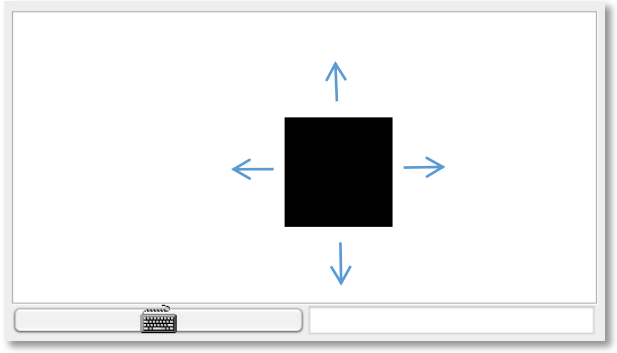
Square class

Square.jack

```
/** Implements a graphical square. */
class Square {

   field int x, y; // screen location of the square's top-left corner
   field int size; // length of this square, in pixels

   ...

   /** Moves the square up by 2 pixels. */
   method void moveUp() {
      if (y > 1) {
         do Screen.setColor(false);
         do Screen.drawRectangle(x, (y + size) - 1, x + size, y + size);
         let y = y - 2;
         do Screen.setColor(true);
         do Screen.drawRectangle(x, y, x + size, y + 1);
      }
      return;
   }

   method void moveDown()  { // similar }

   method void moveLeft()  { // similar }

   method void moveRight() { // similar }

} // class Square
```



Application example: Square Dance



Design: 3 Jack classes:

`Square`: Represents a `Square` object

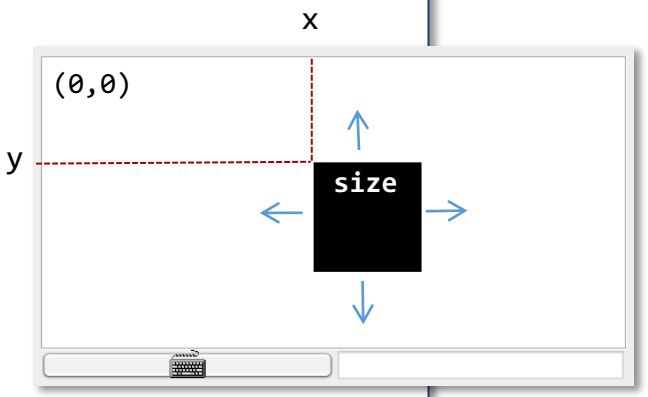
`SquareGame`: Creates a `Square`, then enters a loop that captures the user's input and moves the square / resizes / quits accordingly

`Main:` Creates a `SquareGame`, and launches the game.

SquareGame class

SquareGame.jack

```
/** Implements a square game. */
class SquareGame {

   field Square square; // the square of this game
   field int direction; // the square's current direction:
                        // 0=none, 1=up, 2=down, 3=left, 4=right

   /** Constructs a new Square Game. */
   constructor SquareGame new() {
      let square = Square.new(0, 0, 30);
      let direction = 0;
      return this;
   }

   /** Disposes this game. */
   method void dispose() {
      do square.dispose();
      do Memory.deAlloc(this);
      return;
   }

   /** Moves the square in the current direction. */
   method void moveSquare() {
      if (direction = 1) { do square.moveUp(); }
      if (direction = 2) { do square.moveDown(); }
      if (direction = 3) { do square.moveLeft(); }
      if (direction = 4) { do square.moveRight(); }

      do Sys.wait(5);  // delays the next movement
      return;
   }
   ...
```

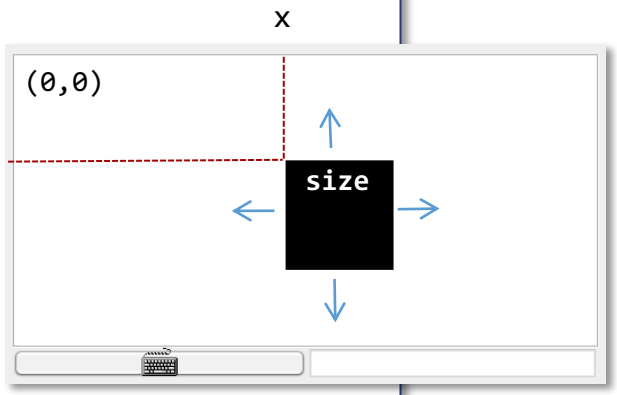


SquareGame class

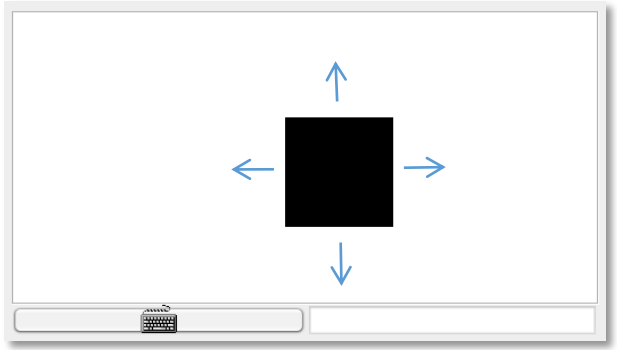
SquareGame.jack

```
/** Implements a square game. */
class SquareGame {

   field Square square; // the square of this game
   field int direction; // the square's current direction:
                        // 0=none, 1=up, 2=down, 3=left, 4=right

   /** Runs the game: handles the user's inputs and moves the square accordingly */
   method void run() {
      var char key;  // the key currently pressed by the user
      var boolean exit;
      let exit = false;

      while (~exit) {
         // waits for a key to be pressed
         while (key = 0) {
            let key = Keyboard.keyPressed();
            do moveSquare();
         }
         if (key = 81)  { let exit = true; }     // q key
         if (key = 90)  { do square.decSize(); } // z key
         if (key = 88)  { do square.incSize(); } // x key
         if (key = 131) { let direction = 1; }   // up arrow
         if (key = 133) { let direction = 2; }   // down arrow
         if (key = 130) { let direction = 3; }   // left arrow
         if (key = 132) { let direction = 4; }   // right arrow

         // waits for the key to be released
         while (~(key = 0)) {
            let key = Keyboard.keyPressed();
            do moveSquare();
         }
      } // while
      return;
   }
}  // SquareGame class
```

typical handling of "keyboard events" in interactive Jack apps



Application example: Square Dance



Main.jack

```
/** Main class of the Square Dance game. */
class Main {

  /** Initializes a new game and starts it. */
  function void main() {
     var SquareGame game;
     let game = SquareGame.new();
     do game.run();
     do game.dispose();
     return;
  }
}
```

Design: 3 Jack classes:

Square: Represents a `Square` object

SquareGame: Creates a `Square`, then enters a loop that captures the user's input and moves the square / resizes / quits accordingly

Main: Creates a `SquareGame`, and launches the game.

Lecture plan

✓ High level programming

- Program example
- Basic language constructs
- Object-based programming
- List processing

Gives a flavor of the language

✓ Jack language specification

- The language
- The operating system

To be used as a technical reference

Application development

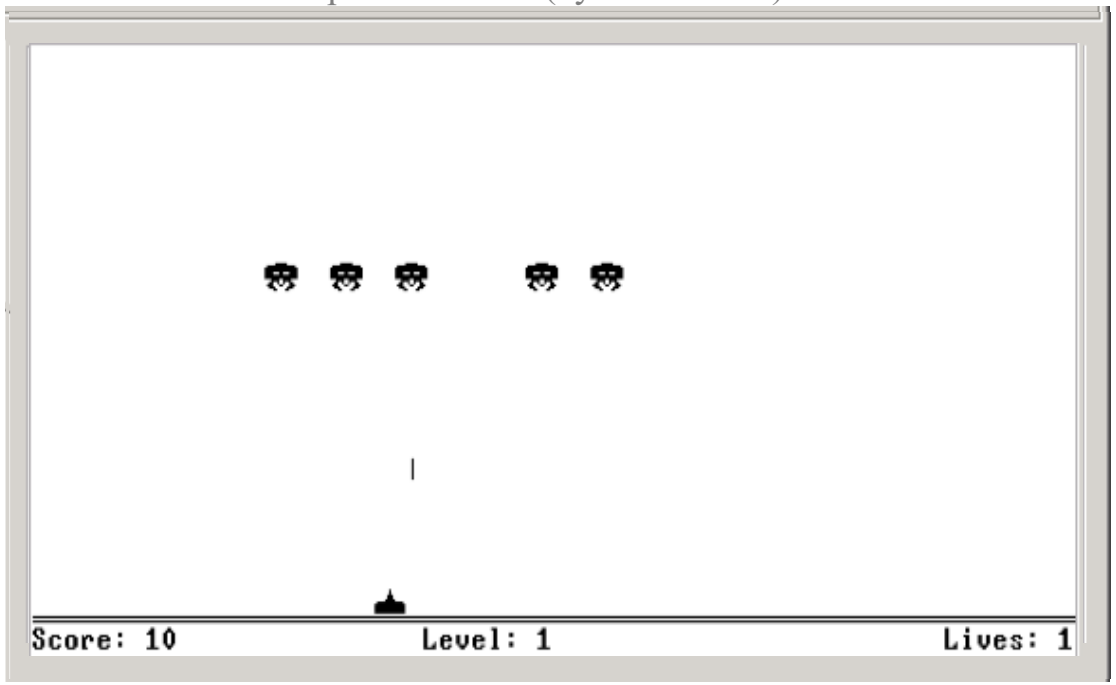
✓ Jack applications

✓ Application example

➡ Graphics optimization

Sprites

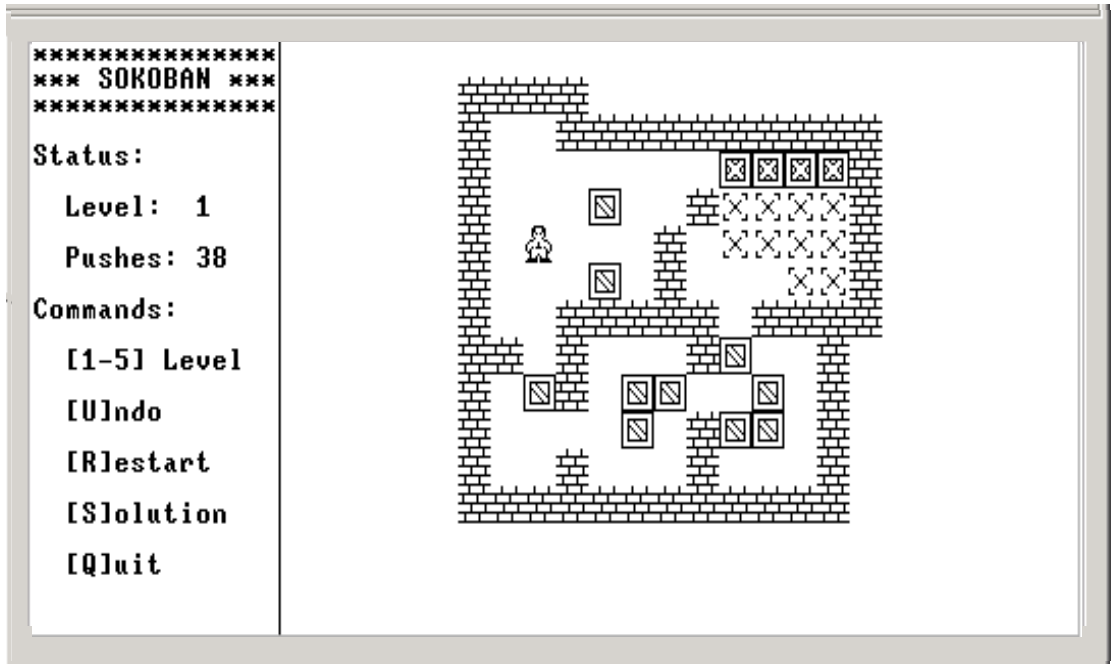
Space Invaders (by Ran Navok)



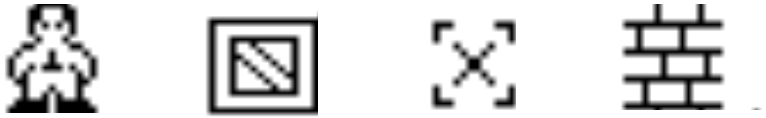
Basic graphical elements (*sprites*):

Sprites

Sokoban (by Golan Parashi)



Basic graphical elements (*sprites*):



Sprites

Sprite

A fixed two-dimensional bitmap, typically integrated into a larger scene

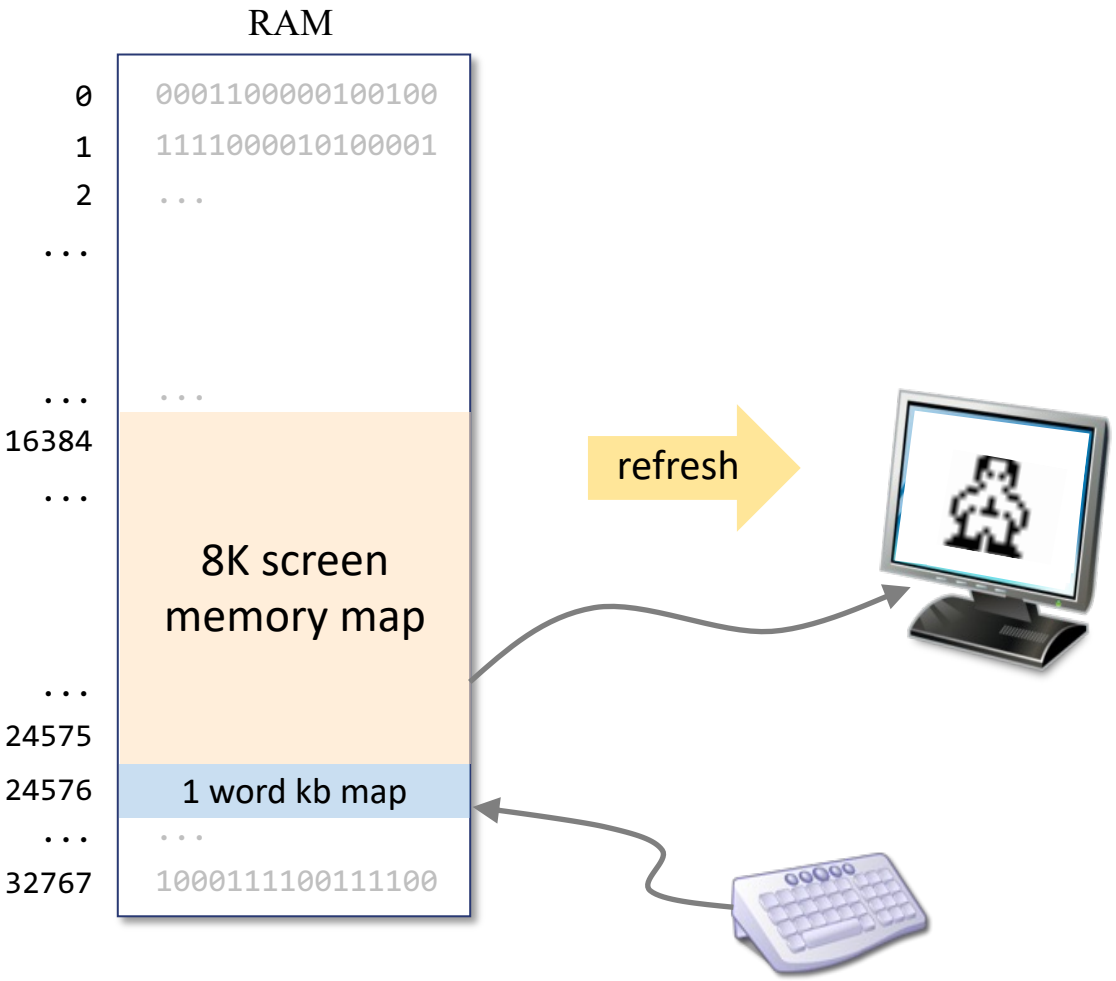
Challenges

- Drawing sprites quickly
- Creating smooth animations

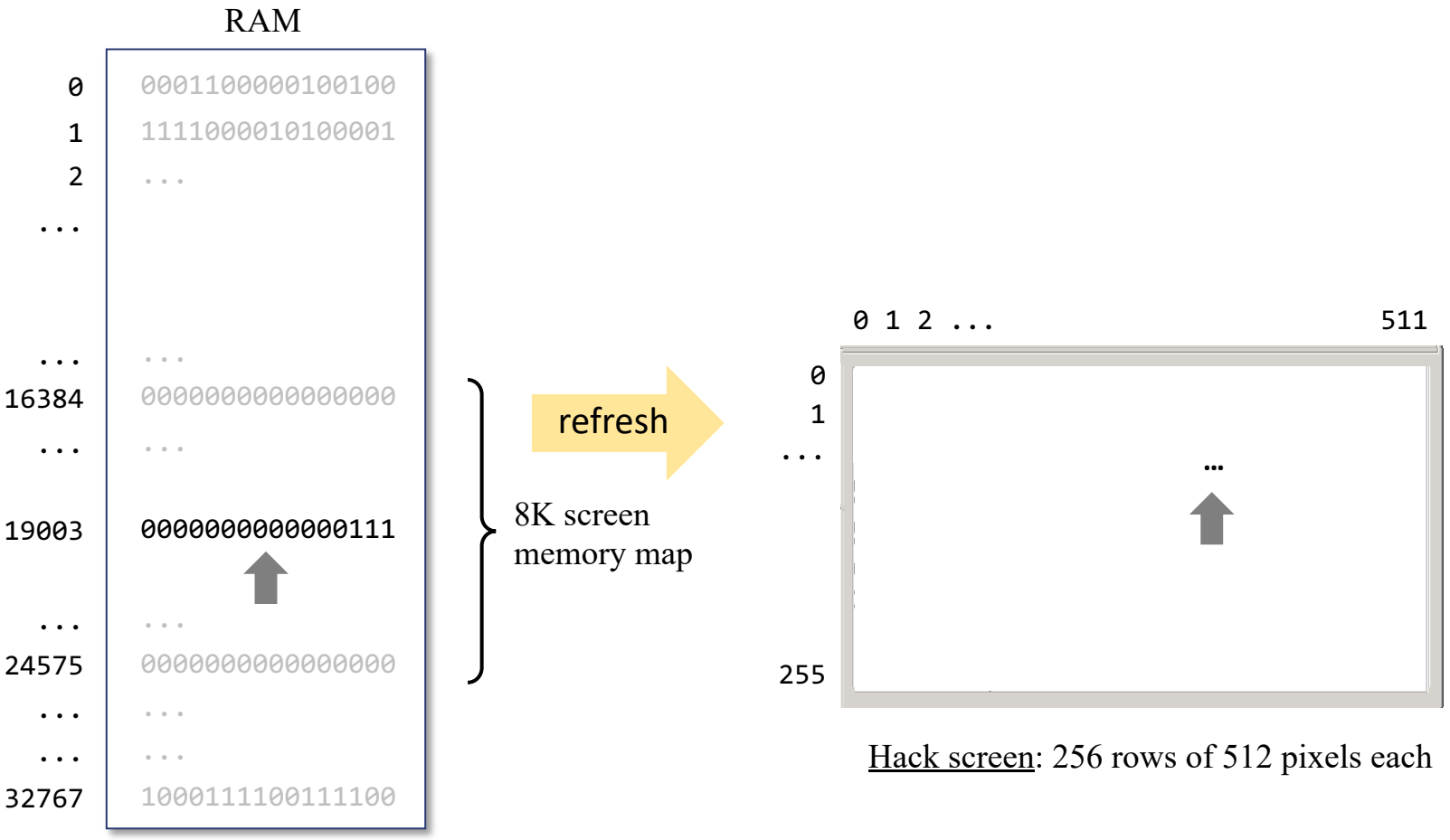
Solutions

- Use the standard OS graphics library (may be slow)
- Write your own graphics functions (can be fast)

Hack output hardware (revisited)

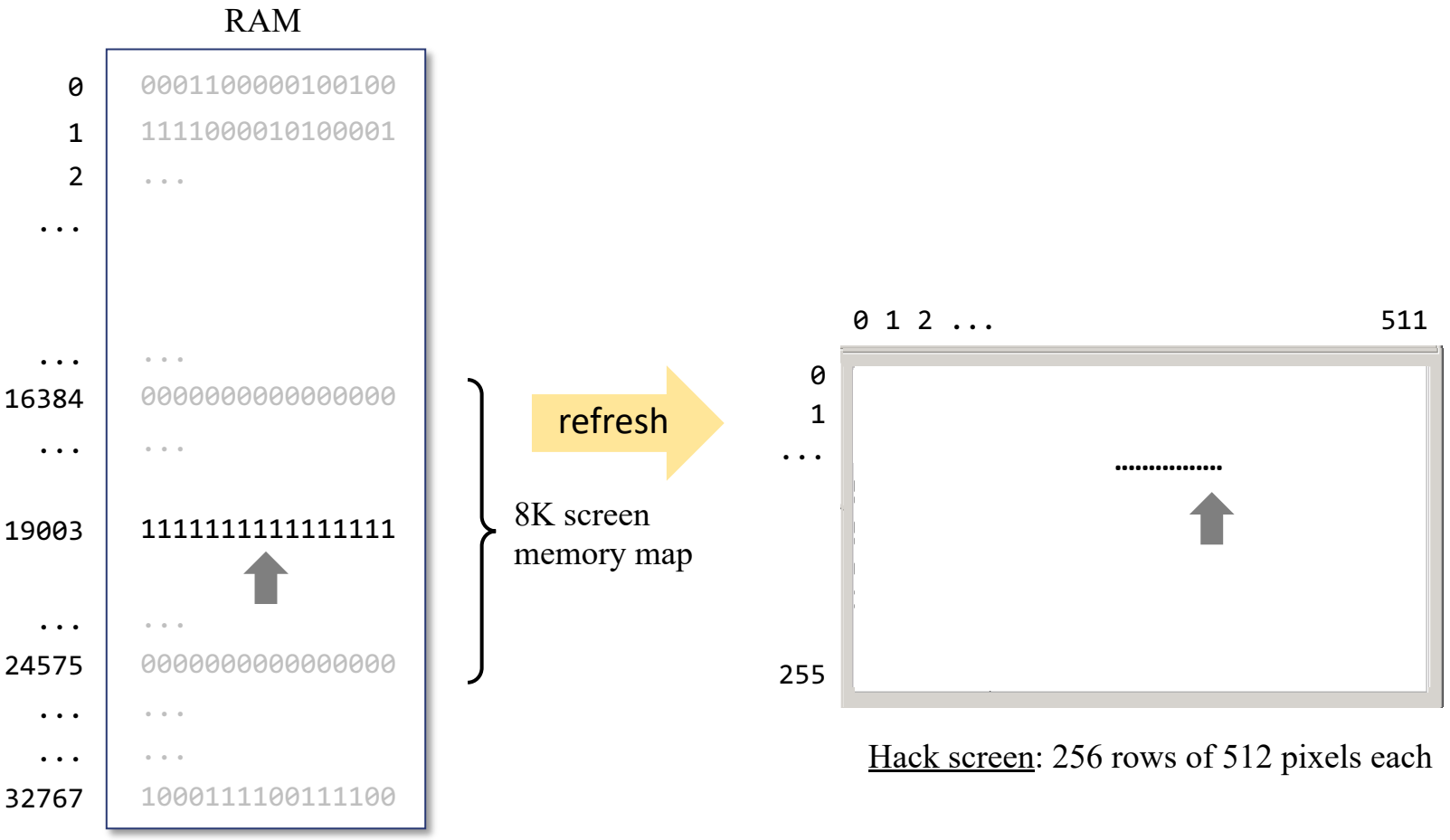


Hack output hardware (revisited)



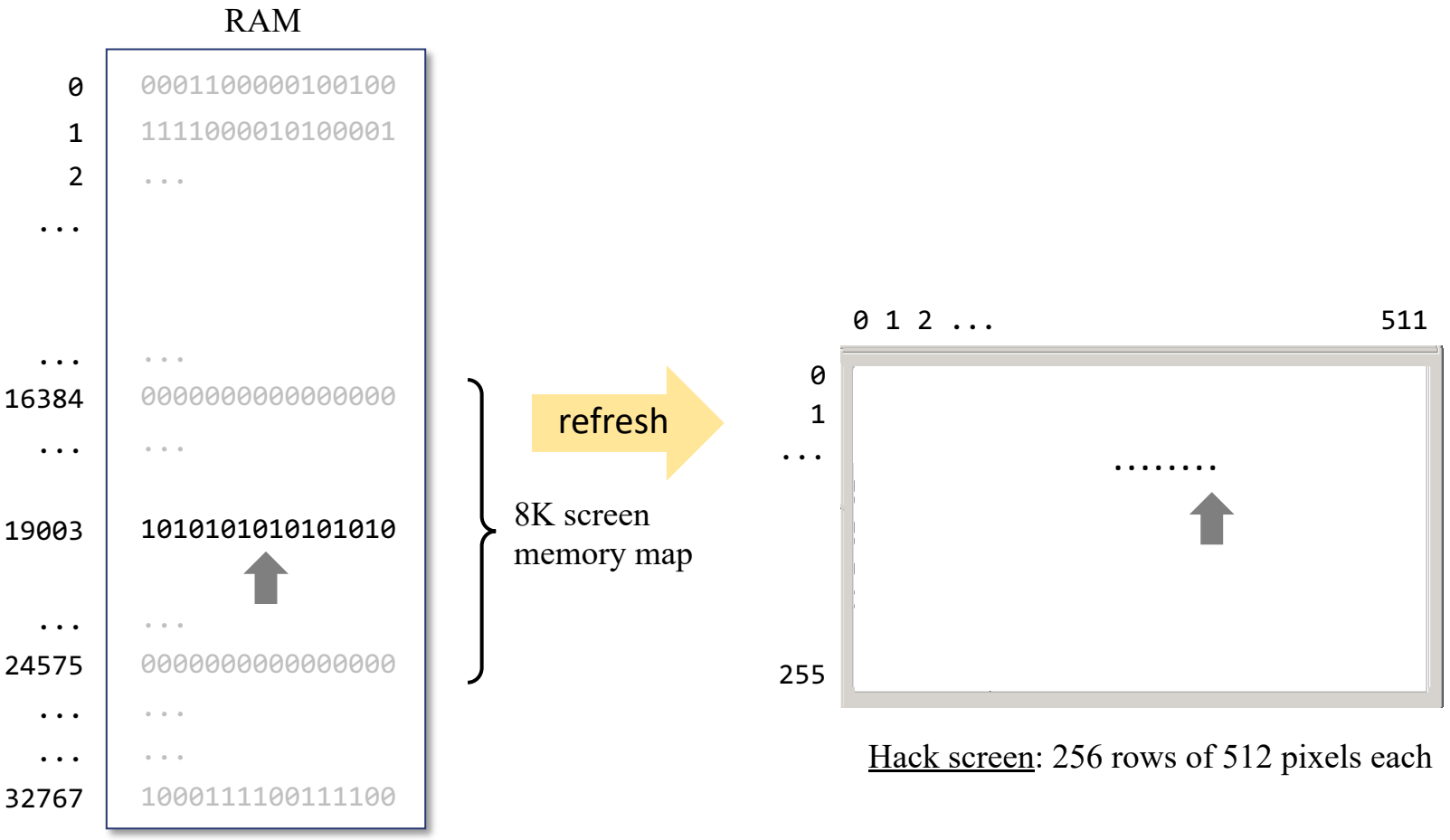
Hack screen: 256 rows of 512 pixels each

Hack output hardware (revisited)



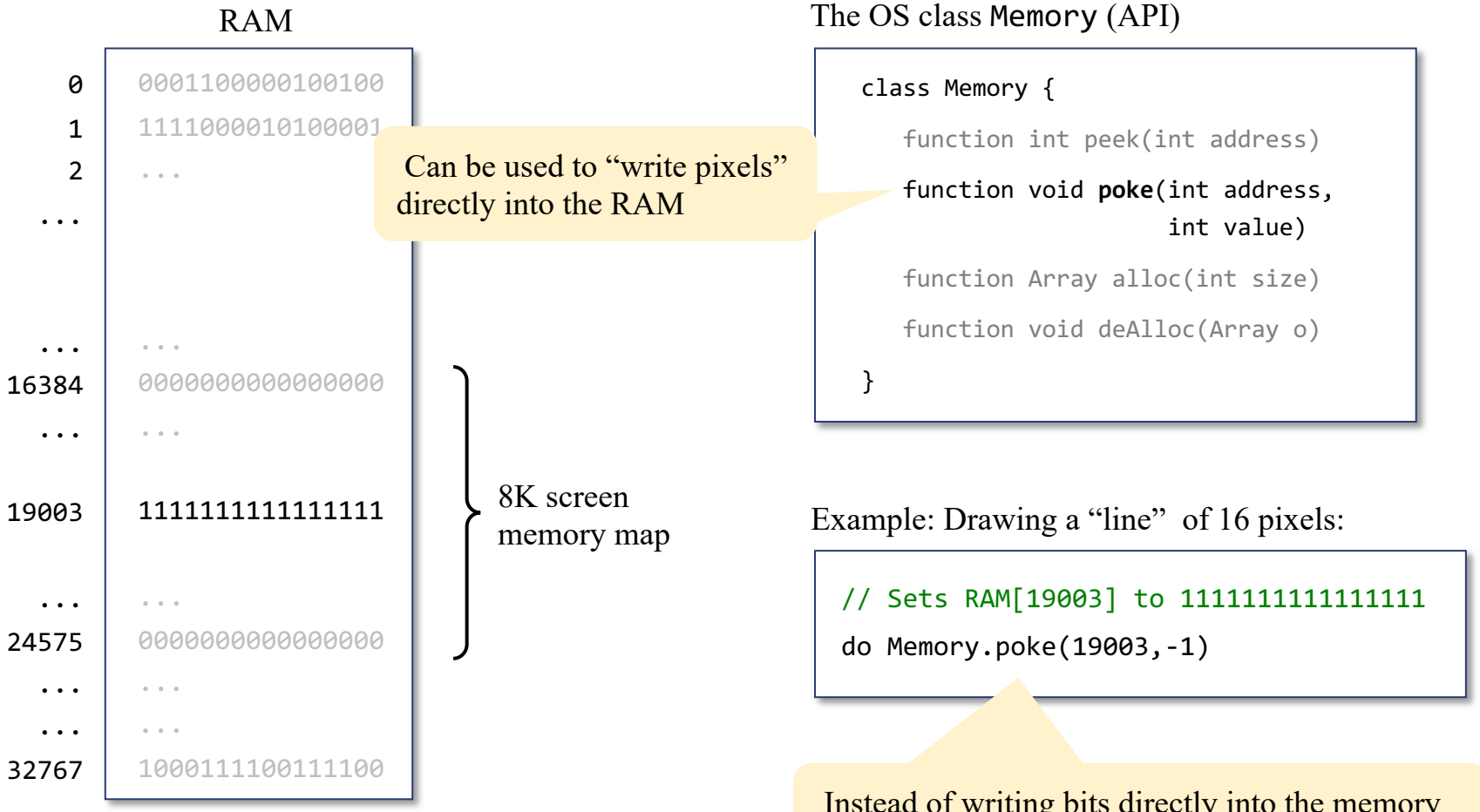
Hack screen: 256 rows of 512 pixels each

Hack output hardware (revisited)



Hack screen: 256 rows of 512 pixels each

Hack output hardware (revisited)



The OS class `Memory` (API)

```
class Memory {
   function int peek(int address)
   function void poke(int address,
                      int value)
   function Array alloc(int size)
   function void deAlloc(Array o)
}
```

Example: Drawing a "line" of 16 pixels:

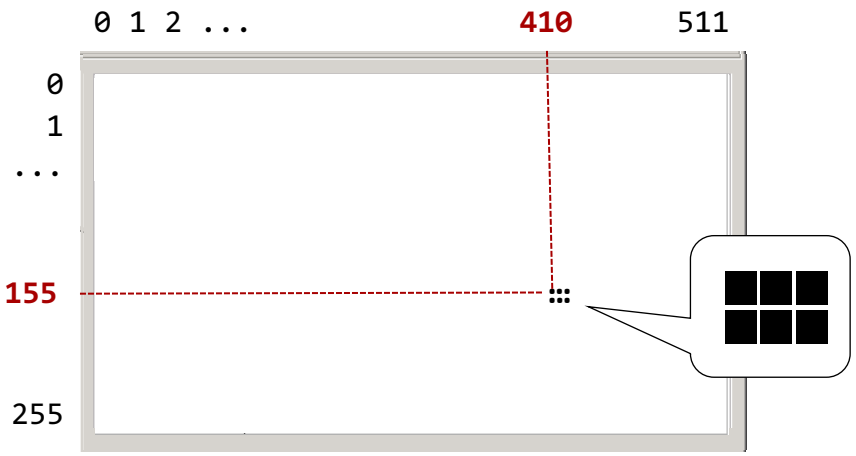
```
// Sets RAM[19003] to 1111111111111111
do Memory.poke(19003,-1)
```

Instead of writing bits directly into the memory map, we can use the OS graphics library (`Screen`)

Standard drawing (may be slow)

The OS class `Screen` (API)

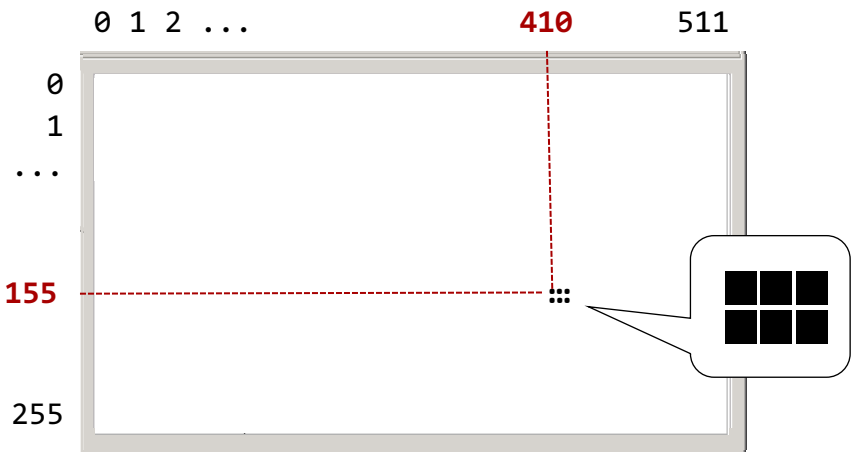
```
Class Screen {
   function void clearScreen()
   function void setColor(boolean b)
   function void drawPixel(int x, int y)
   function void drawLine(int x1, int y1,
                          int x2, int y2)
   function void drawRectangle(int x1, int y1,
                               int x2, int y2)
   function void drawCircle(int x, int y, int r)
}
```



Standard drawing (may be slow)

The OS class `Screen` (API)

```
Class Screen {
   function void clearScreen()
   function void setColor(boolean b)
   function void drawPixel(int x, int y)
   function void drawLine(int x1, int y1,
                          int x2, int y2)
   function void drawRectangle(int x1, int y1,
                               int x2, int y2)
   function void drawCircle(int x, int y, int r)
}
```



Jack code (option 1)

```
// draws the image
do Screen.drawPixel(410,155);
do Screen.drawPixel(411,155);
do Screen.drawPixel(412,155);
do Screen.drawPixel(410,156);
do Screen.drawPixel(411,156);
do Screen.drawPixel(412,156);
```

Jack code (option 2)

```
// draws the image
do Screen.drawLine(410,155,412,155);
do Screen.drawLine(410,156,412,156);
```

Jack code (option 3)

```
// draws the image
do Screen.drawRectangle(410,155,412,156);
```

Standard drawing (may be slow)

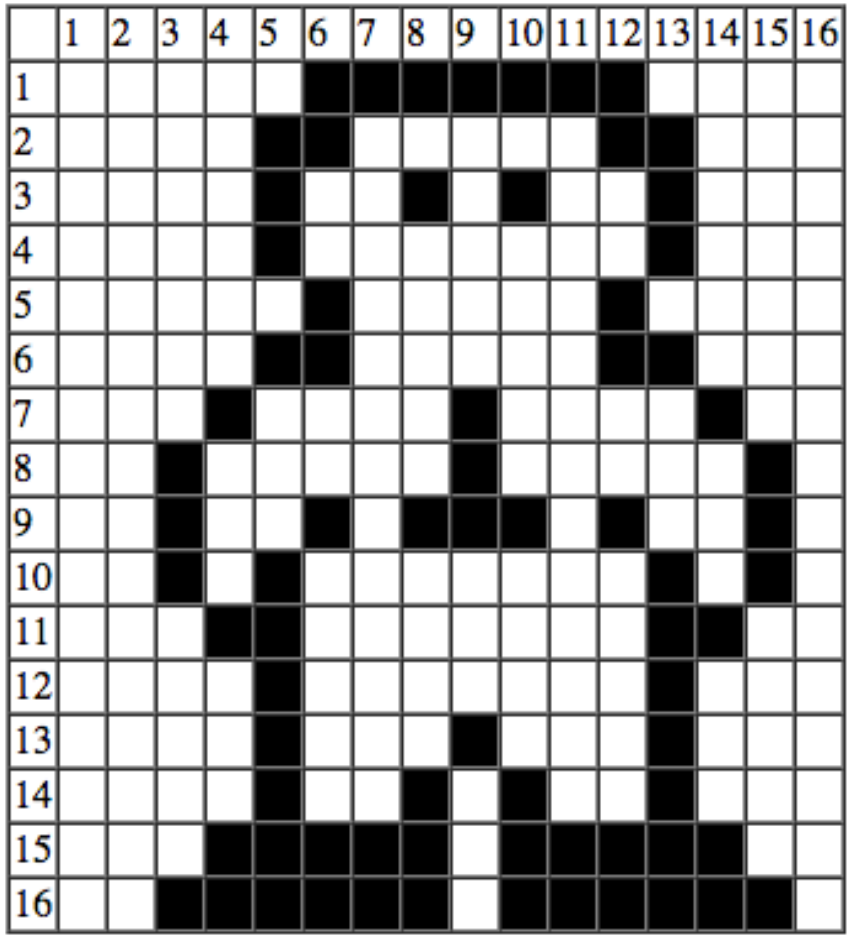


Image drawing code:

```
// Draws the top row
do Screen.drawPixel(6,1);
do Screen.drawPixel(7,1);
...
do Screen.drawPixel(12,1);
...
// Draws the bottom row
do Screen.drawPixel(3,16);
...
do Screen.drawPixel(15,16);
```

Efficiency:

75 pixel drawing operations

OS implementation of `drawPixel(x,y)`

// sets pixel (*x*,*y*) to black / white:
address = 32 * *y* + *x* / 16
value = `Memory.peek[`16384 + *address*`]`
set the (*x* % 16)*th* bit of *value* to 0 or 1
do `Memory.poke(`*address*`,`*value*`)`

More about this in chapter 12 (OS)

Standard drawing (may be slow)

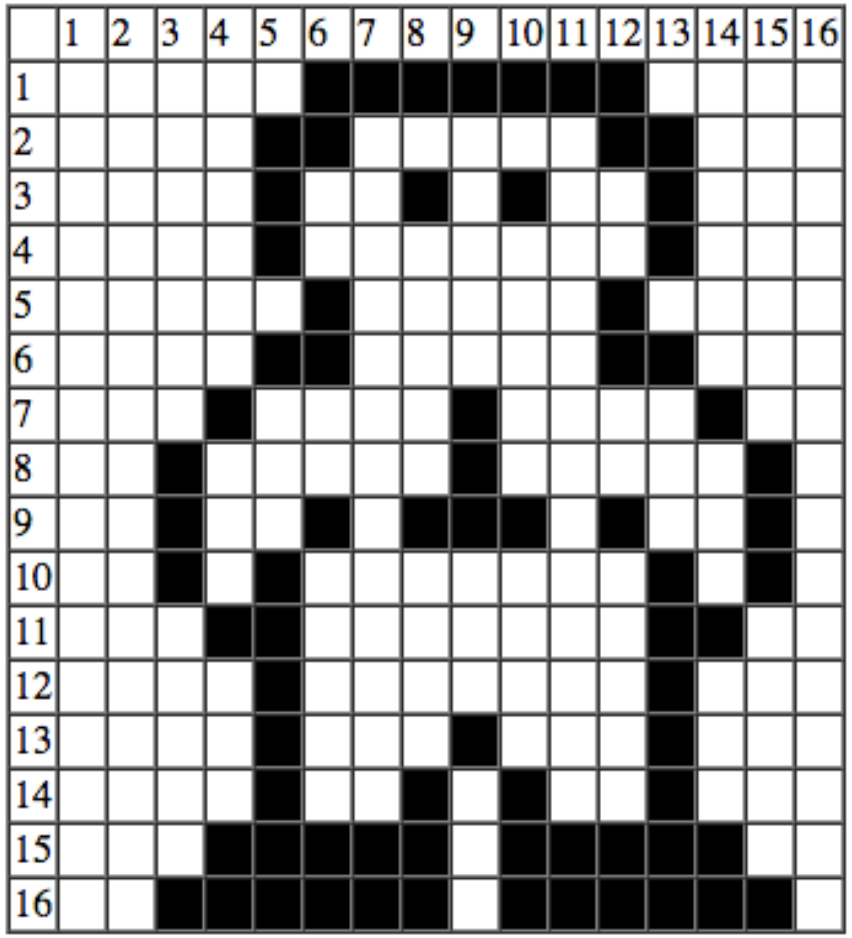


Image drawing code:

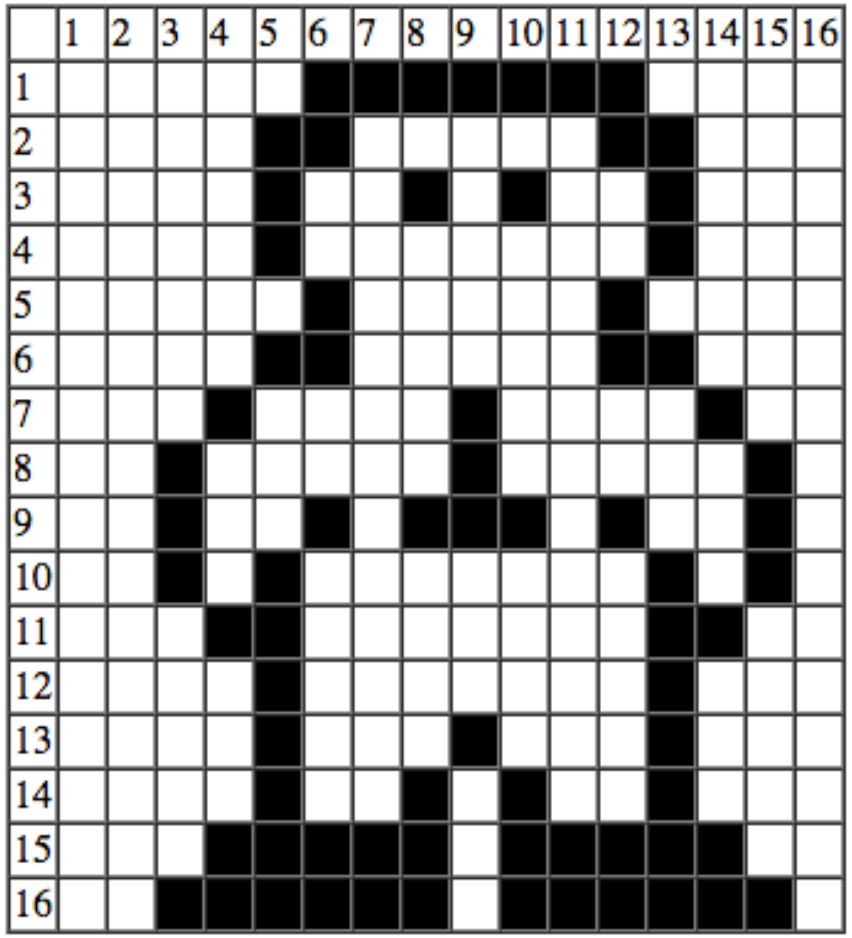
```
// Draws the top row
do Screen.drawPixel(6,1);
do Screen.drawPixel(7,1);
...
do Screen.drawPixel(12,1);
...
// Draws the bottom row
do Screen.drawPixel(3,16);
...
do Screen.drawPixel(15,16);
```

Efficiency:

75 pixel drawing operations

Can we do better?

Custom drawing (optimized)



```
0000111111100000 = 4064
0001100000110000 = 6192
0001001010010000 = 4752
...
...
0111111011111100 = 32508
```

Custom drawing (optimized)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
1						■	■	■	■	■	■	■					4064
2					■	■						■	■				6192
3					■			■		■			■				4752
4					■								■				...
5						■						■					
6					■	■						■	■				
7				■					■					■			
8			■						■						■		
9			■			■		■	■	■		■			■		
10			■		■								■		■		
11				■	■								■	■			
12					■								■				
13					■				■				■				
14					■			■		■			■				
15				■	■	■	■	■		■	■	■	■	■			...
16			■	■	■	■	■	■		■	■	■	■	■	■		32508

Image drawing code:

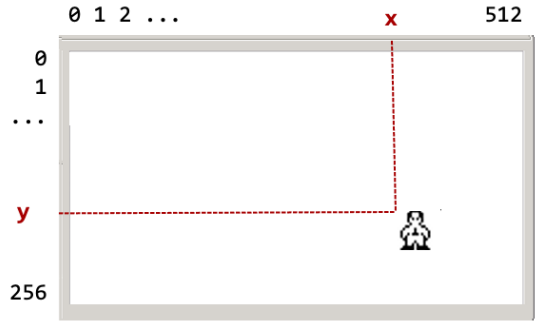
```
// Draws the sprite
do Memory.poke(addr0, 4064);
do Memory.poke(addr1, 6192);
do Memory.poke(addr2, 4752);
...
do Memory.poke(addr15,32508);
```

Efficiency:

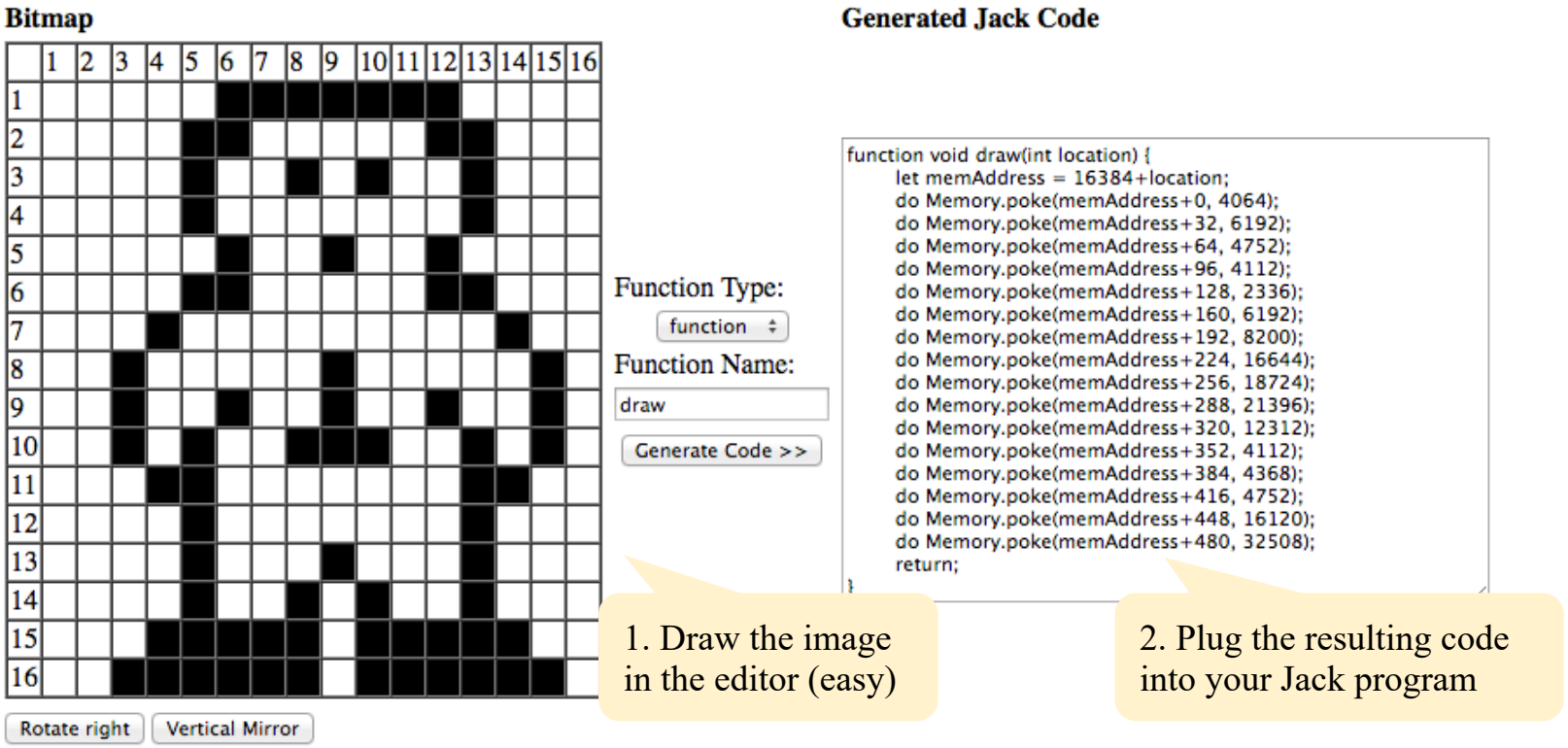
16 memory write operations

We still have to compute:

`addr0, addr1, ..., addr15` (as a function of x, y)



Bitmap editor



Developed by Golan Parashi (available in `nand2tetris/projects/09`)

Best Practice:

- For simple graphics, use the OS library `Screen`
- For high-performance sprites, use the bitmap editor, and `Memory.poke`

Perspective

Jack is a simple language,

Featuring essential elements of:

- Procedural programming
- OO programming

Limitations

- Few control structures
- Some peculiar syntax
- No inheritance

Motivation: a minimal language that can be implemented by a simple compiler

Data types

- Primitive type system
- Weakly typed

Motivation: to give the programmer full control, especially for writing the OS.

Chapter 9

High Level Language

These slides support chapter 9 of the book
The Elements of Computing Systems
By Noam Nisan and Shimon Schocken
MIT Press